



SECOND SEMESTER SESSION 2024/2025 (A242)

STIAK1113 PROGRAMMING 2

GROUP PROJECT :

COMPUTER SHOP MANAGEMENT SYSTEM

LECTURER NAME : DR. SHAFINAH FARVIN BINTI PACKEER MOHAMED

GROUP CLASS : GROUP B

SUBMISSION DATE : 26 / 06 / 2025

GROUP MEMBERS :

	NAME	MATRIC NUMBER
1.	LEONG WEI CHONG	299202
2.	TANG HUI YI	299390
3.	SHARWIN A/L PARAMESWARAN	295026

TABLE OF CONTENTS

- 1. INTRODUCTION**
- 2. SYSTEM OVERVIEW**
- 3. PROBLEM ANALYSIS**
- 4. UML DIAGRAM**
- 5. SYSTEM FUNCTIONS**
- 6. TARGET USERS**
- 7. USER INTERFACE GUIDE**
- 8. FUNCTION DESCRIPTIONS**
- 9. TECHNICAL ARCHITECTURE**
- 10. DATA MANAGEMENT**
- 11. CONCLUSION**
- 12. SOURCE CODE**

1. INTRODUCTION

The Computer Shop Management System is a comprehensive Java-based application designed to streamline and automate the operations of computer retail businesses. This system addresses the critical need for efficient inventory management, sales tracking, customer relationship management, and financial reporting in the computer retail industry.

1.1. Project Background

In today's rapidly evolving technology market, computer retailers face numerous challenges in managing their operations effectively. The traditional manual methods of inventory tracking, sales recording, and customer management are not only time-consuming but also prone to errors and inefficiencies. This system was developed to provide a robust, user-friendly solution that automates these processes while maintaining accuracy and providing valuable business insights.

1.2. System Purpose

The Computer Shop Management System serves as a centralized platform for managing all aspects of a computer retail business, from product inventory to sales transactions and customer relationships. The system is designed to be intuitive, efficient, and scalable to meet the growing needs of computer retailers.

2. SYSTEM OVERVIEW

The Computer Shop Management System is built using Java with Swing for the graphical user interface, providing both console-based and GUI-based interaction modes. The system follows object-oriented programming principles and implements a modular architecture for better maintainability and extensibility.

2.1. Key Features

- Product Management: Complete inventory control with add, update, delete, and view operations
- Sales Management: Comprehensive sales tracking with cart functionality and discount management
- Customer Membership: Member registration and benefits system
- Financial Reporting: Profit analysis and sales summaries
- Data Persistence: File-based data storage for reliability
- Dual Interface: Both console and GUI interfaces for user flexibility

2.2. Technology Stack

- Programming Language: Java
- User Interface: Java Swing (GUI), Console Interface
- Data Storage: Text file-based storage system
- Architecture: Object-Oriented Programming

3. PROBLEM ANALYSIS

3.1. Manual System Limitations

Traditional computer shop operations face several critical challenges:

3.1.1. Inventory Management Issues

- Manual Stock Tracking: Paper-based inventory records are prone to errors and difficult to update in real-time
- Stock Discrepancies: Manual counting often leads to inaccurate stock levels, resulting in overselling or understocking
- Time-Consuming Updates: Adding new products or updating existing ones requires significant manual effort
- Limited Search Capabilities: Finding specific products in manual records is time-consuming and inefficient

3.1.2. Sales Process Inefficiencies

- Slow Transaction Processing: Manual calculation of prices, discounts, and totals slows down customer service
- Error-Prone Calculations: Manual arithmetic increases the risk of pricing errors and customer dissatisfaction
- Limited Sales Tracking: Difficulty in maintaining comprehensive sales records for analysis
- Complex Discount Management: Manual application of various discount tiers and membership benefits is cumbersome

3.1.3. Customer Management Challenges

- Membership Tracking: Manual membership records are difficult to maintain and verify
- Limited Customer Insights: Lack of systematic customer data makes it difficult to provide personalized service
- Communication Gaps: No systematic way to track customer

preferences or purchase history

3.1.4. Financial Reporting Difficulties

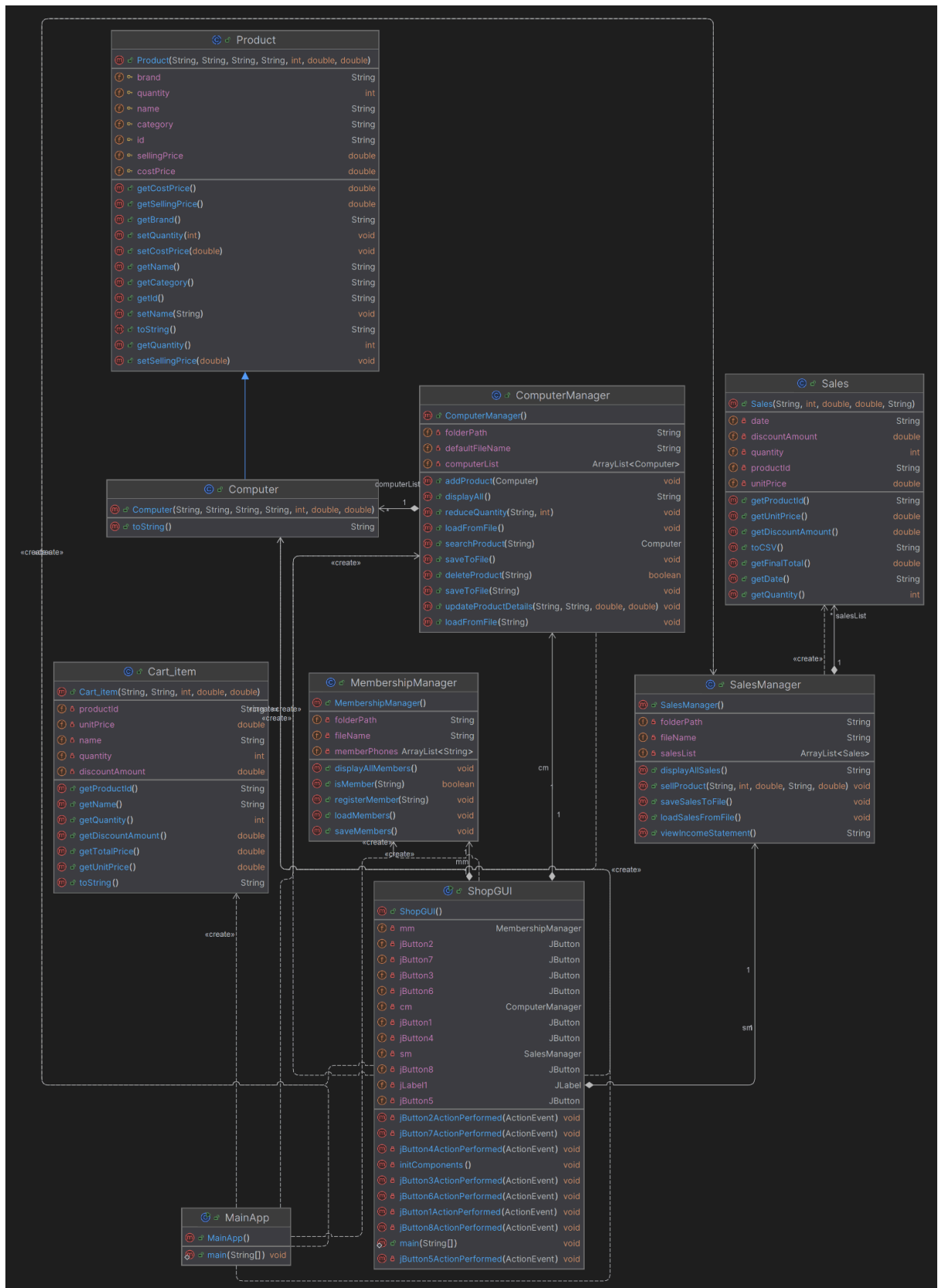
- Manual Calculations: Profit calculations and financial summaries require extensive manual work
- Limited Analysis: Difficulty in generating comprehensive business reports for decision-making
- Data Inconsistency: Manual records often contain inconsistencies that affect financial accuracy

3.2. Need for Automation

The implementation of an automated system becomes essential due to:

- Accuracy Requirements: Automated calculations eliminate human errors in pricing and inventory management
- Efficiency Demands: Faster transaction processing improves customer service and operational efficiency
- Scalability Needs: As the business grows, manual systems become increasingly inadequate
- Competitive Advantage: Automated systems provide better customer service and operational insights
- Data-Driven Decisions: Systematic data collection enables better business decision-making

4. UML DIAGRAM



5. SYSTEM FUNCTIONS

5.1. Core Functionalities

5.1.1. Product Management

- Add New Products: Register new computer products with complete specifications
- Update Product Information: Modify existing product details, pricing, and stock levels
- Delete Products: Remove discontinued or obsolete products from inventory
- View Product Catalog: Display comprehensive product listings with search capabilities

5.1.2. Sales Management

- Shopping Cart System: Multi-item transaction processing with cart functionality
- Discount Management: Automated application of tiered discounts and membership benefits
- Transaction Recording: Complete sales history with detailed transaction records
- Stock Reduction: Automatic inventory updates after successful sales

5.1.3. Membership Management

- Member Registration: New customer membership enrollment
- Membership Verification: Quick verification of existing member status
- Benefit Application: Automatic application of member

discounts and privileges

5.1.4. Reporting and Analytics

- Sales Reports: Comprehensive sales history and transaction details
- Profit Analysis: Financial summaries and profit calculations
- Inventory Status: Current stock levels and product availability

5.2. Advanced Features

5.2.1. Discount System

- Tiered Discounts: Automatic application based on purchase amount
 - ◆ 5% discount for purchases above RM 5,000
 - ◆ 8% discount for purchases above RM 10,000
- Membership Benefits: Additional 2% discount for registered members
- Cumulative Savings: Transparent display of total savings for customers

5.2.2. Data Persistence

- File-Based Storage: Reliable data storage using text files
- Automatic Backup: Data is automatically saved after each transaction
- Data Recovery: System loads existing data on startup

6. TARGET USERS

6.1. Shop Owners/Managers

- Role: Overall system administration and business oversight
- Functions Used: All system functions including reporting and analytics
- Benefits: Complete business control, financial insights, and operational efficiency

6.2. Sales Staff/Cashiers

- Role: Day-to-day sales operations and customer service
- Functions Used: Product viewing, sales processing, membership management
- Benefits: Faster transaction processing, accurate calculations, improved customer service

6.3. Inventory Managers

- Role: Product management and stock control
- Functions Used: Product addition, updates, deletion, and inventory monitoring
- Benefits: Accurate stock tracking, efficient product management, automated updates

7. USER INTERFACE GUIDE

7.1. Home Screen

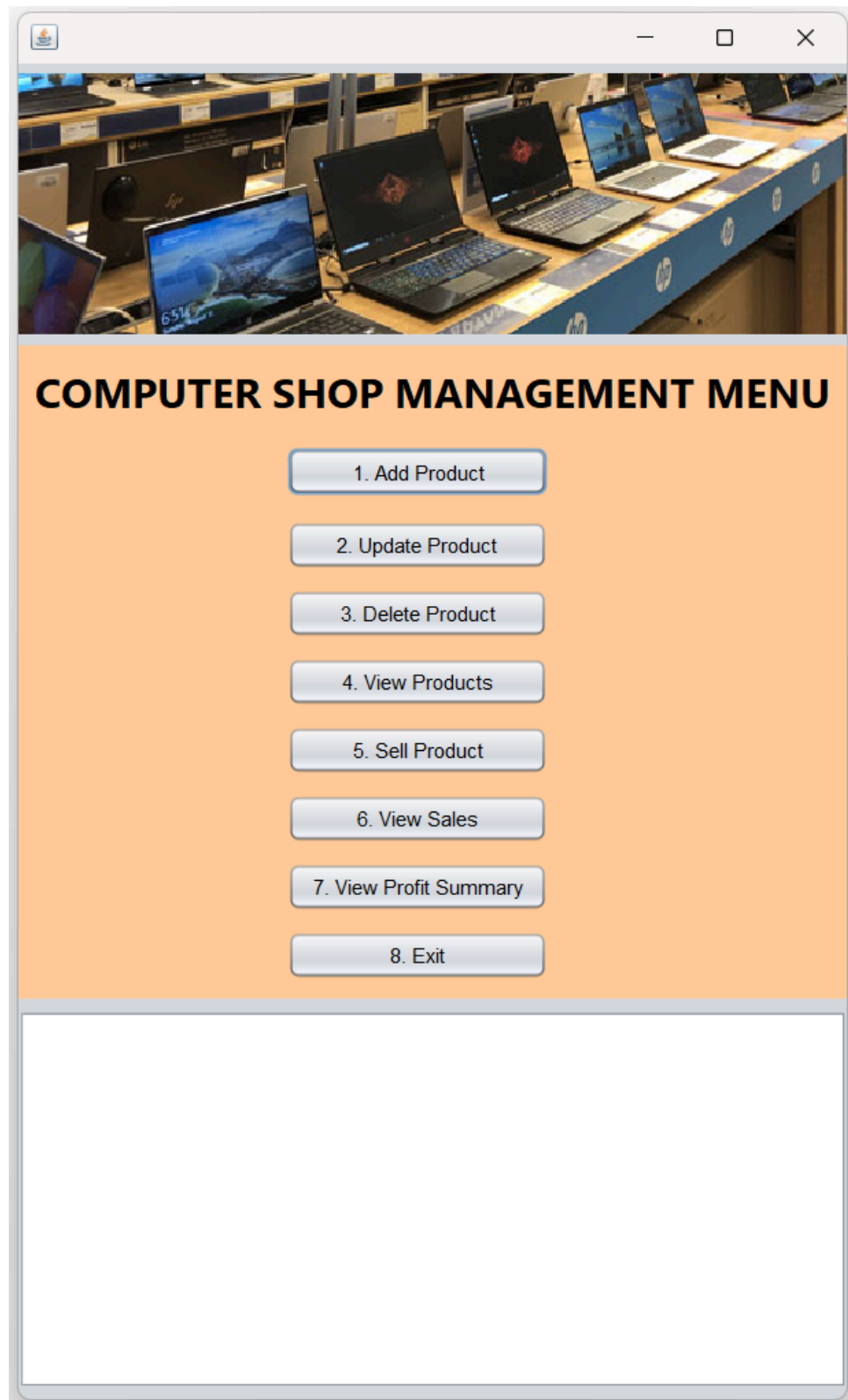


Image 1.0 : Home screen

7.2. Add Product Button

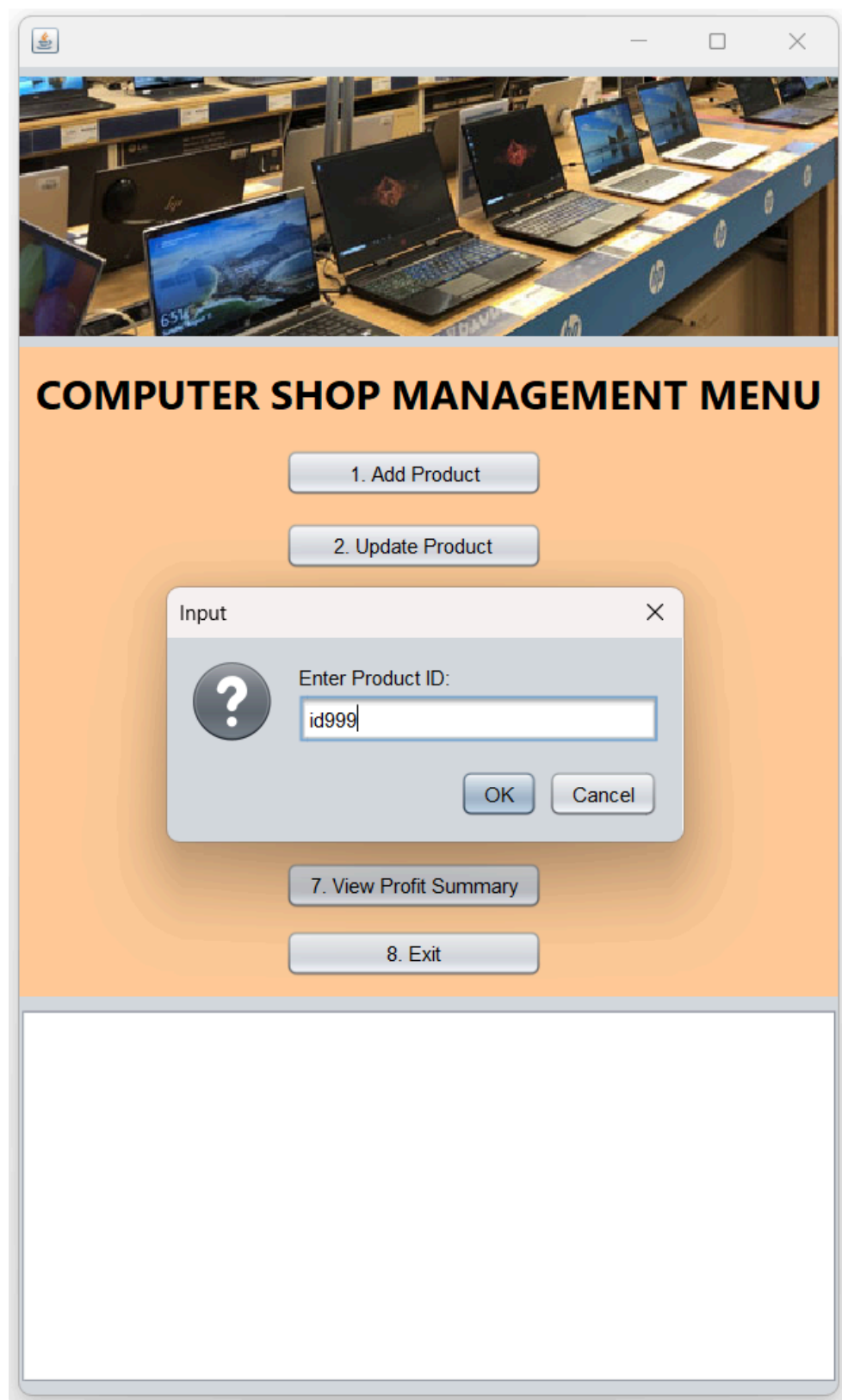


Image 2.0 : Enter Product ID

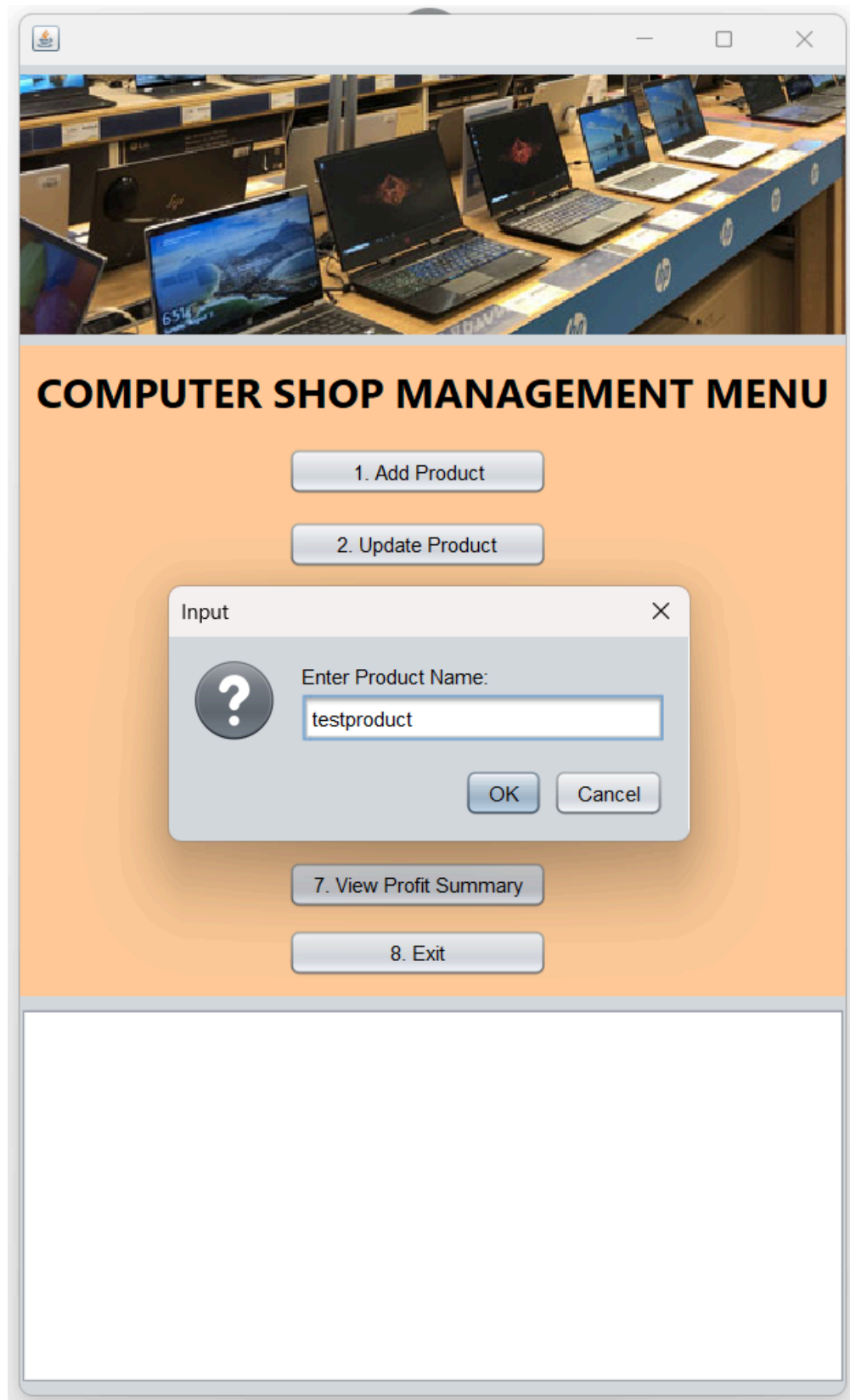


Image 2.1 : Enter Product Name

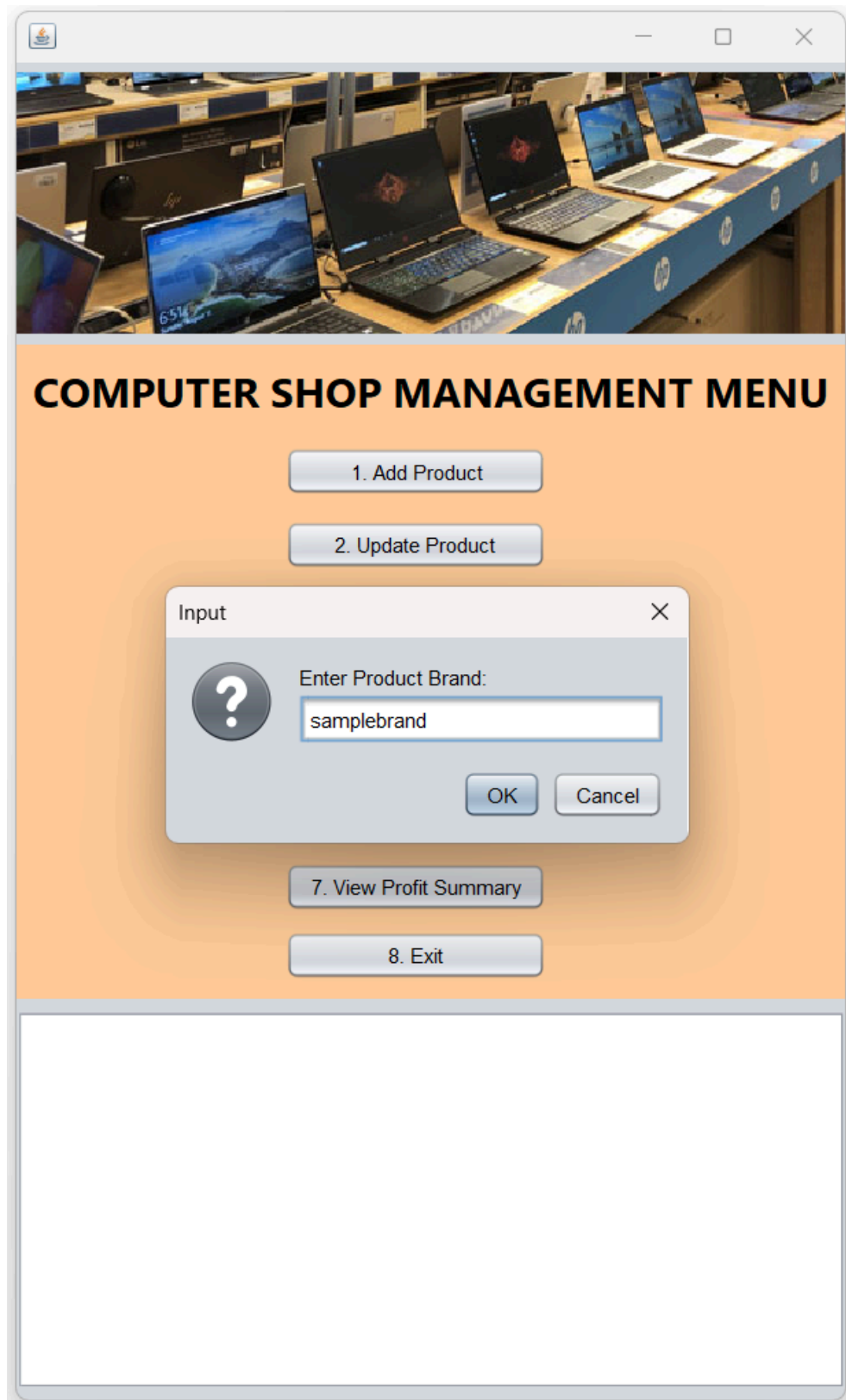


Image 2.2 : Enter Product Brand

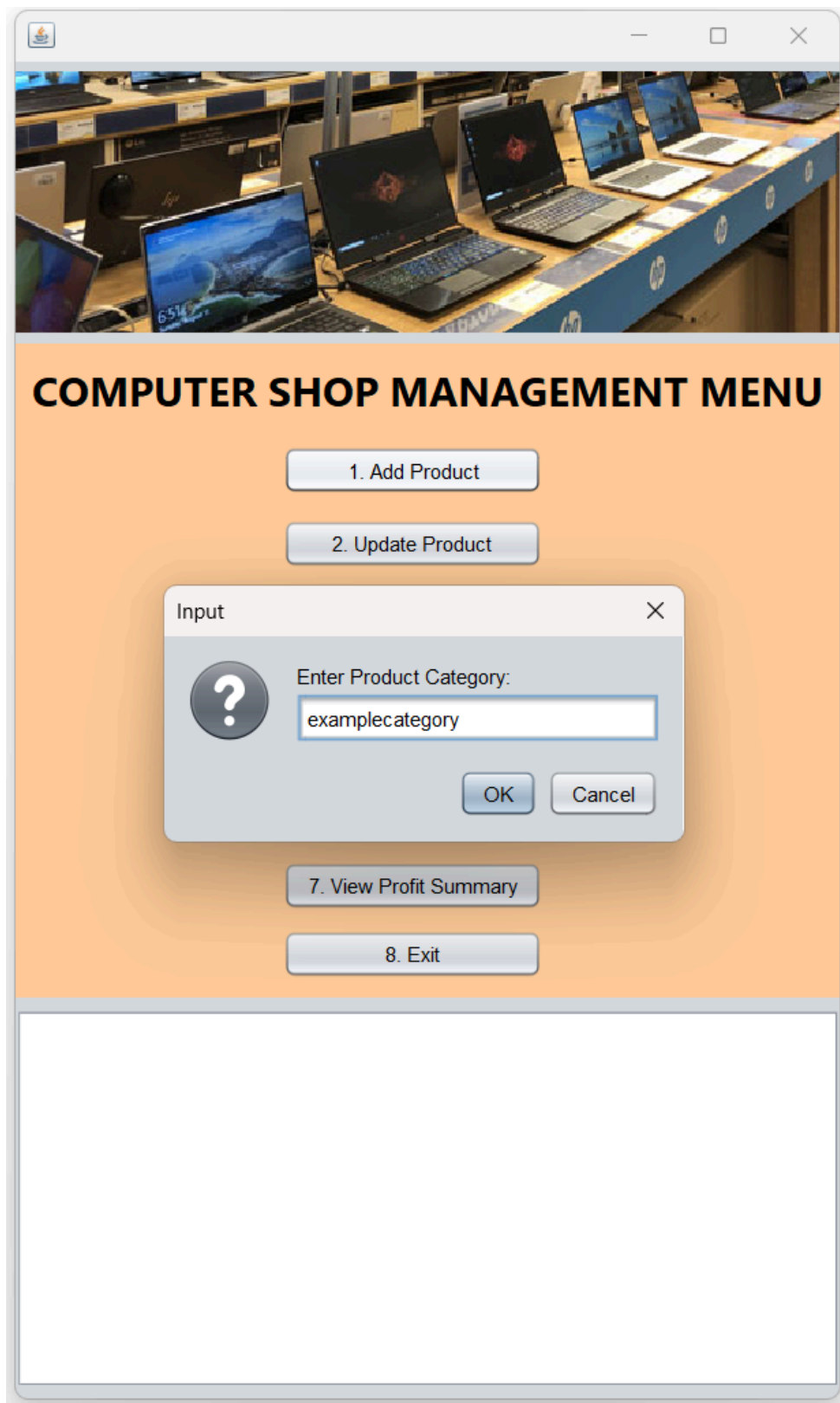


Image 2.3 : Enter Product Category



Image 2.4 : Enter Quantity

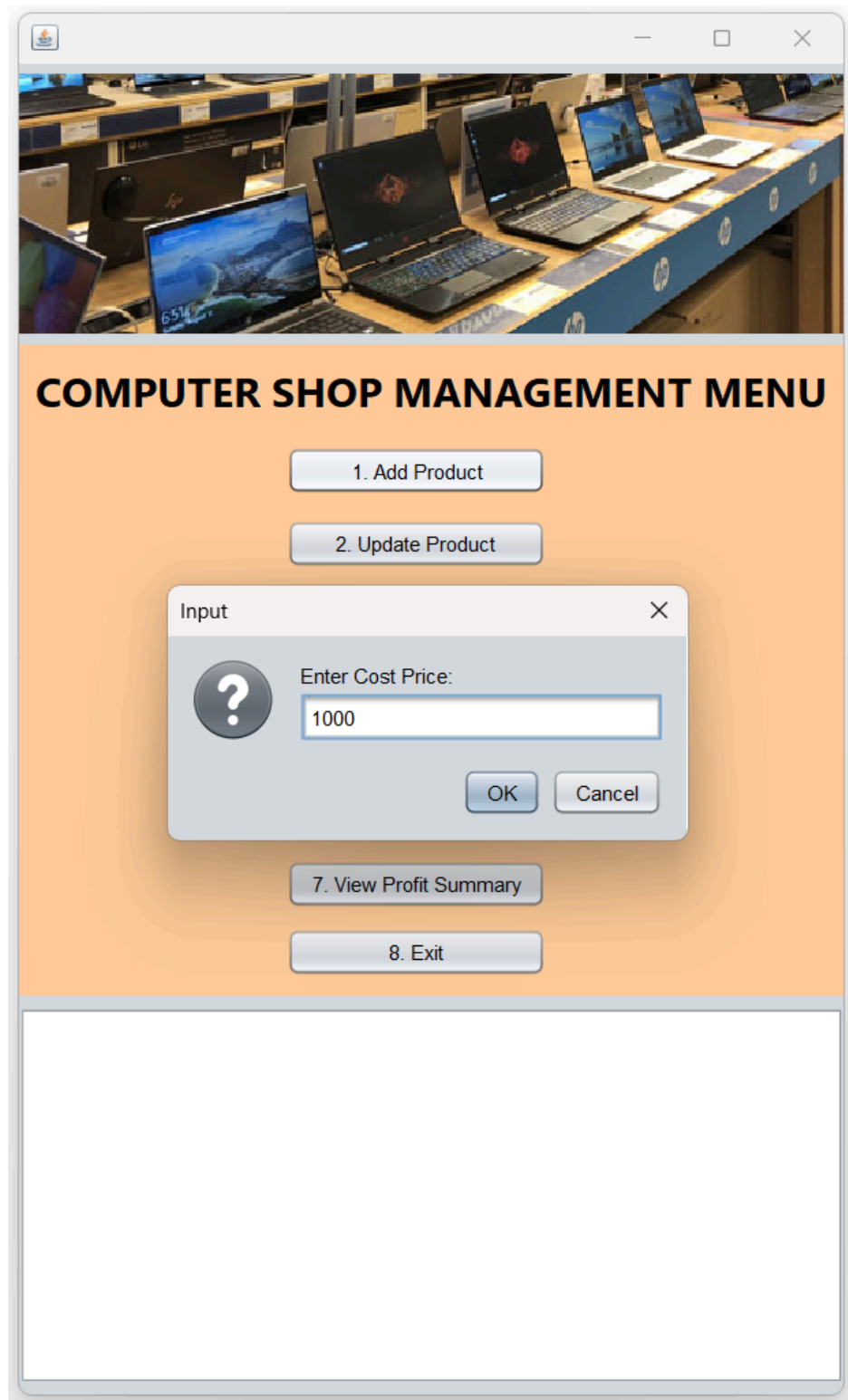


Image 2.5 : Enter Cost Price

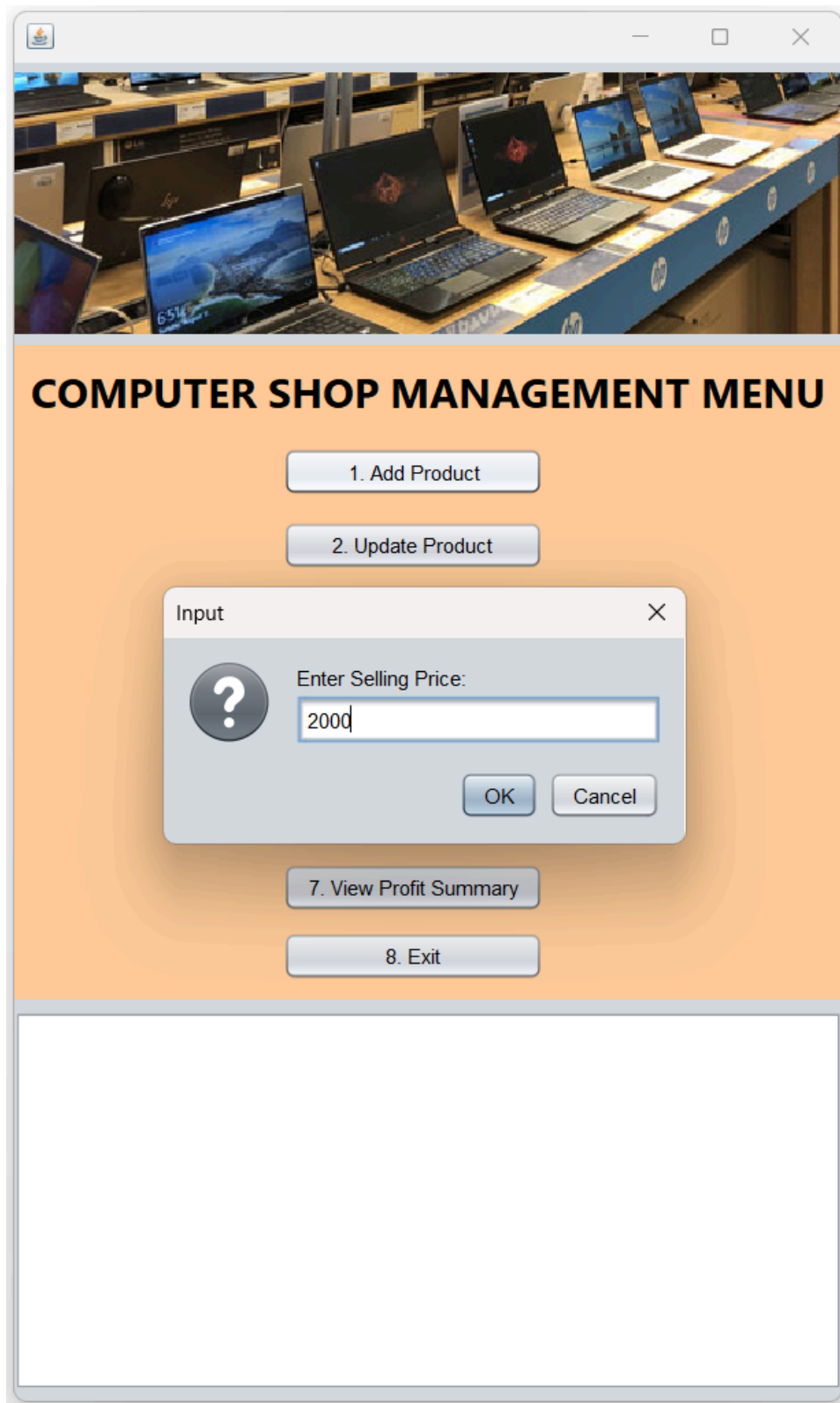


Image 2.6 : Enter Selling Price

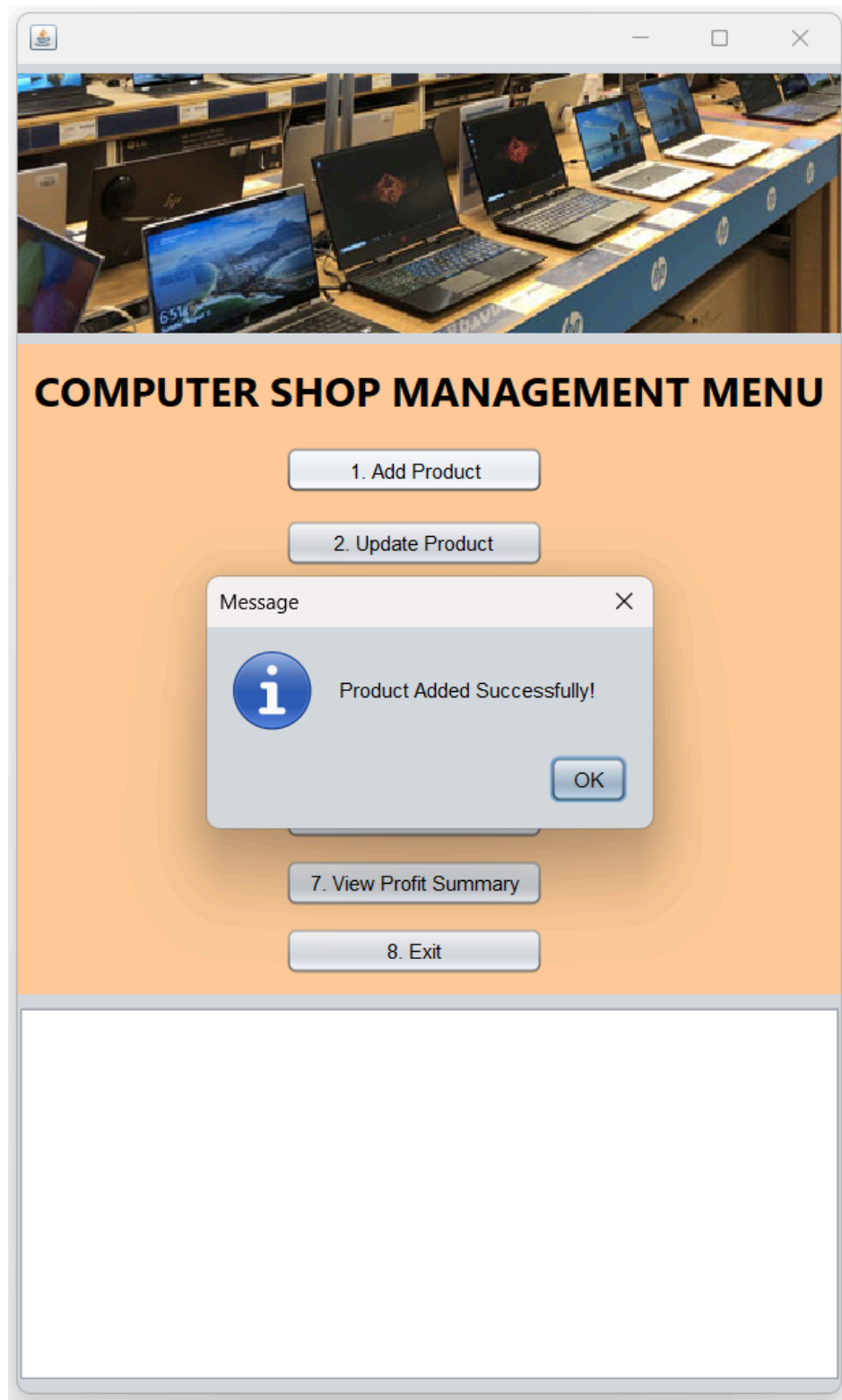


Image 2.7 : Product Added Successfully

7.3. Update Product Button

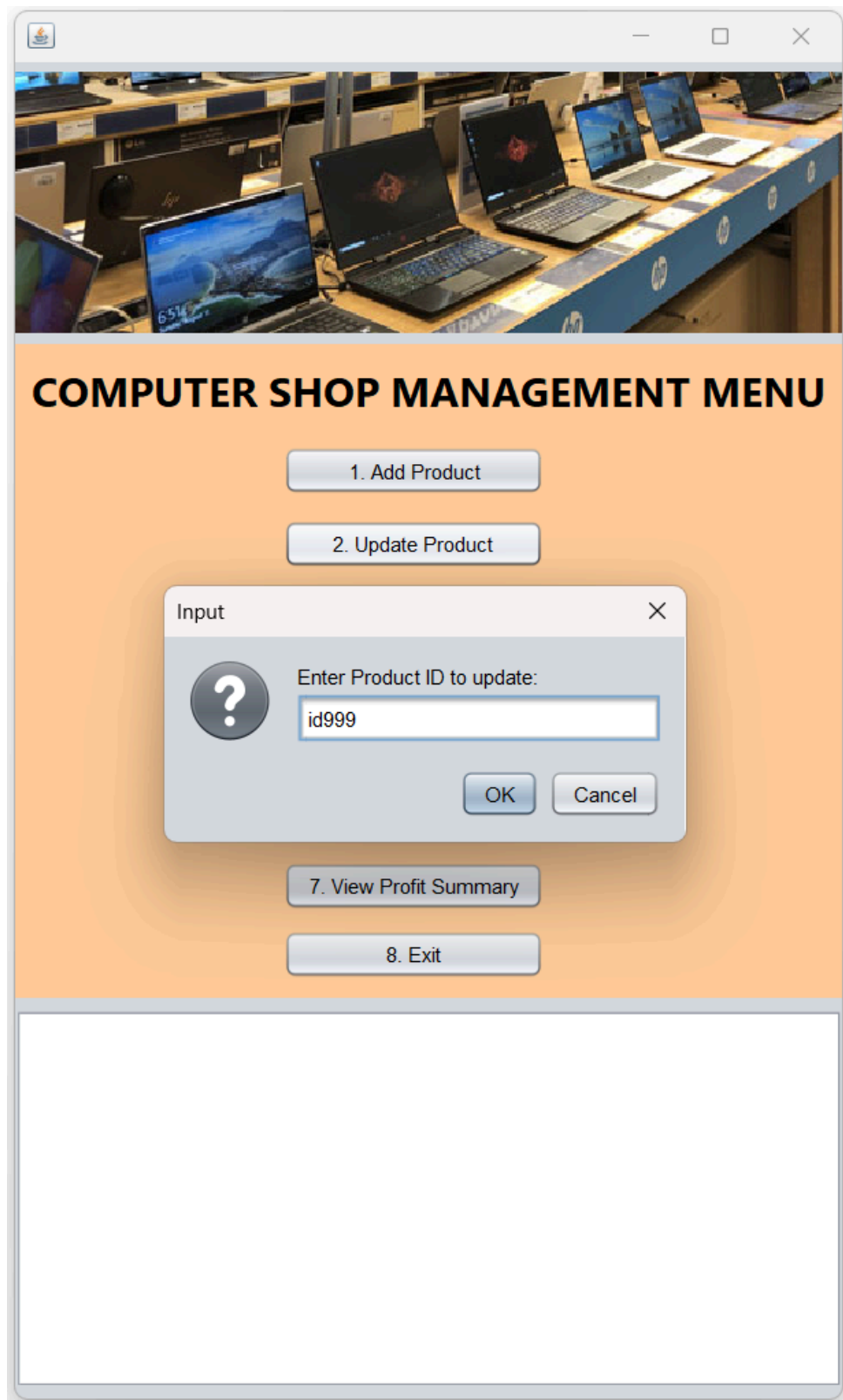


Image 3.0 : Enter Product ID to update

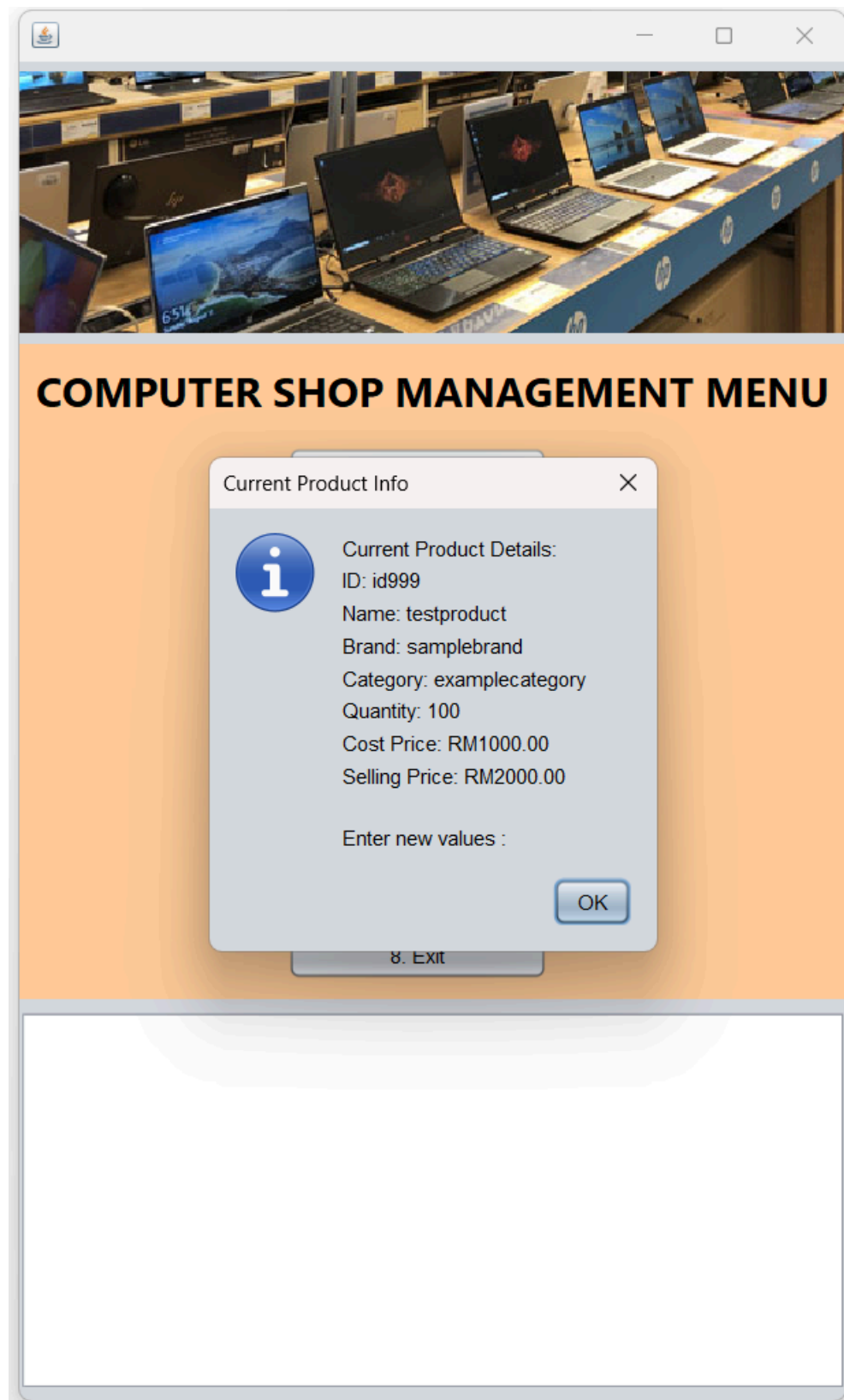


Image 3.1 : Current Product Info

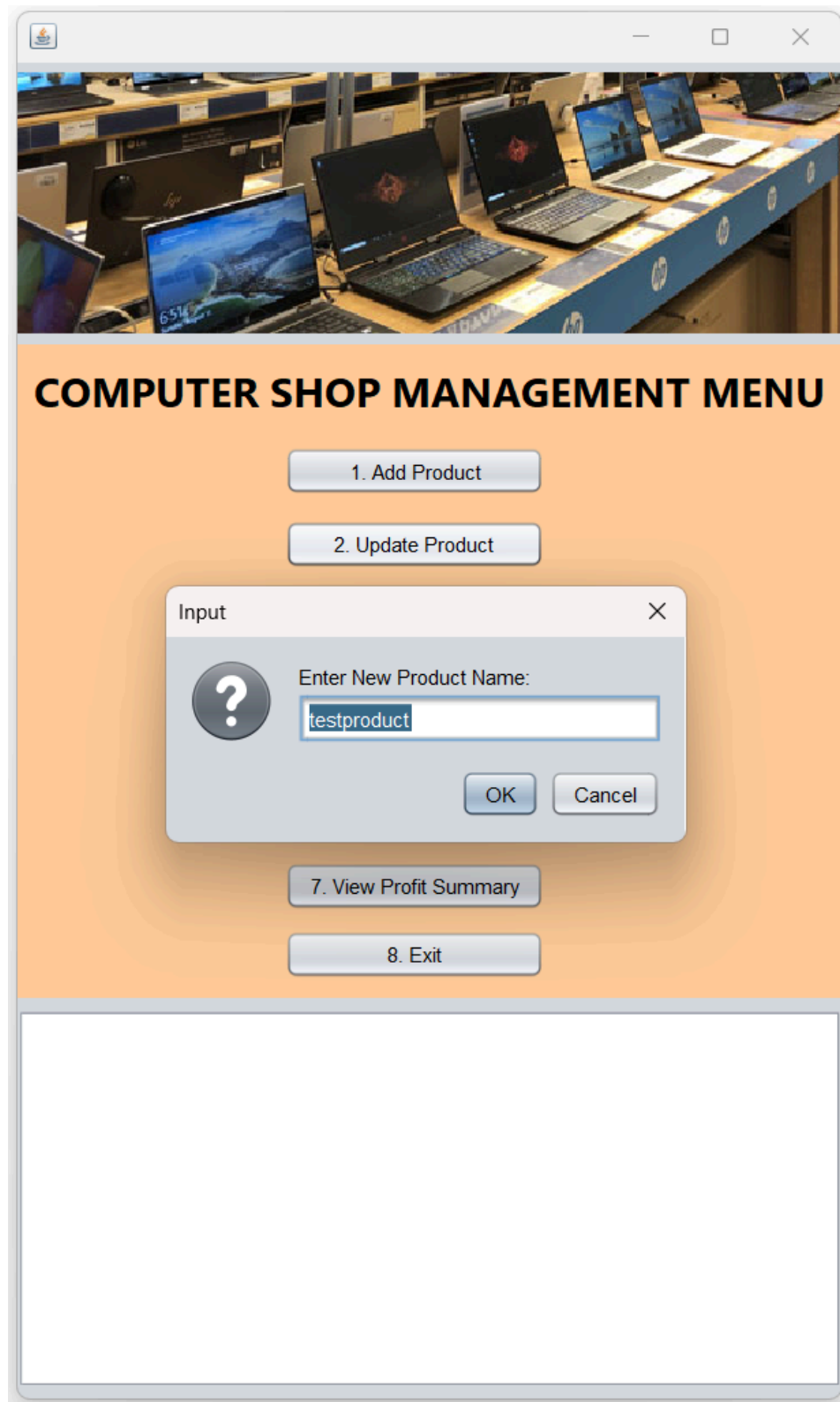


Image 3.2 : Enter New Product Name (Unchanged)

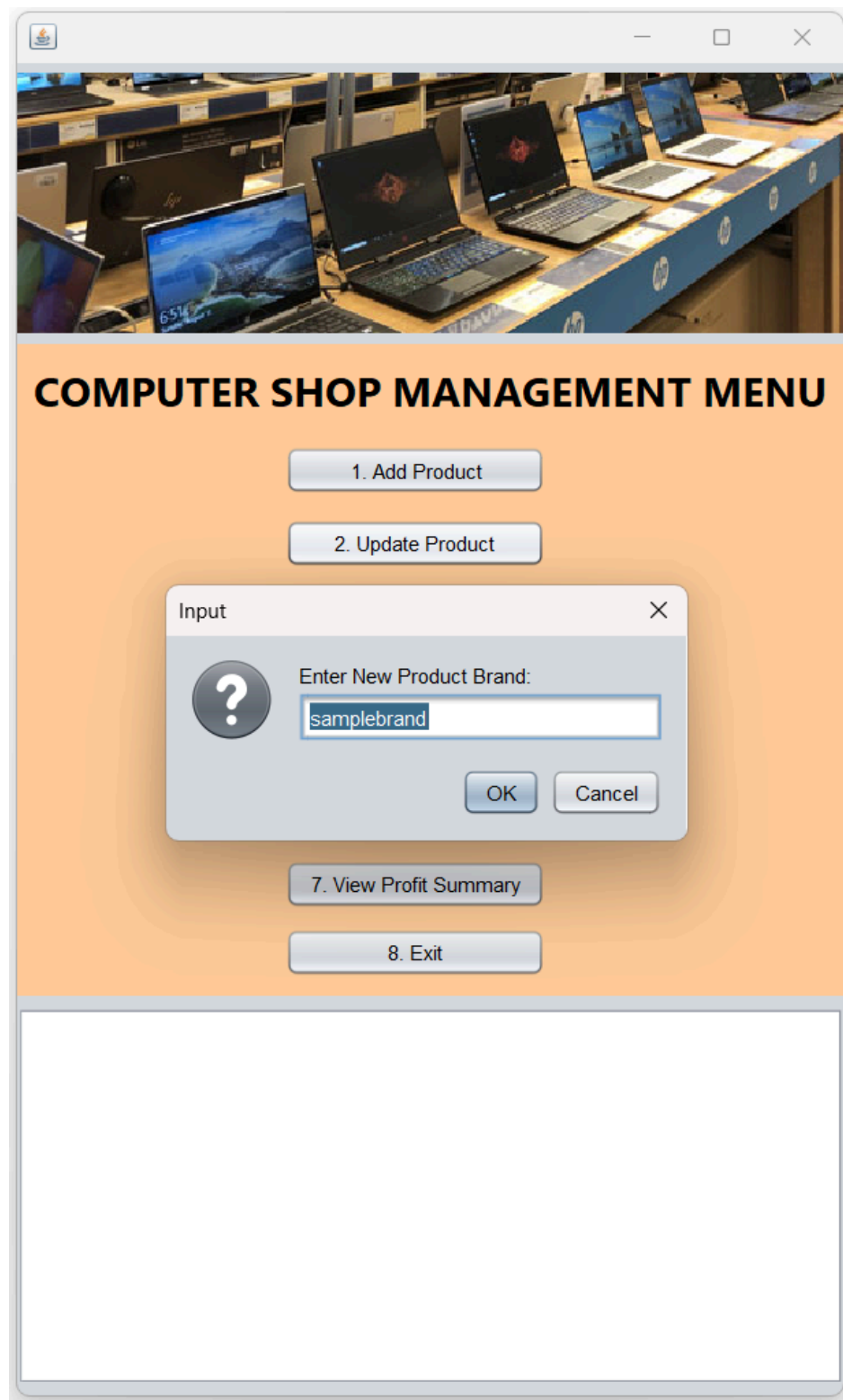


Image 3.3 : Enter New Product Brand (Unchanged)

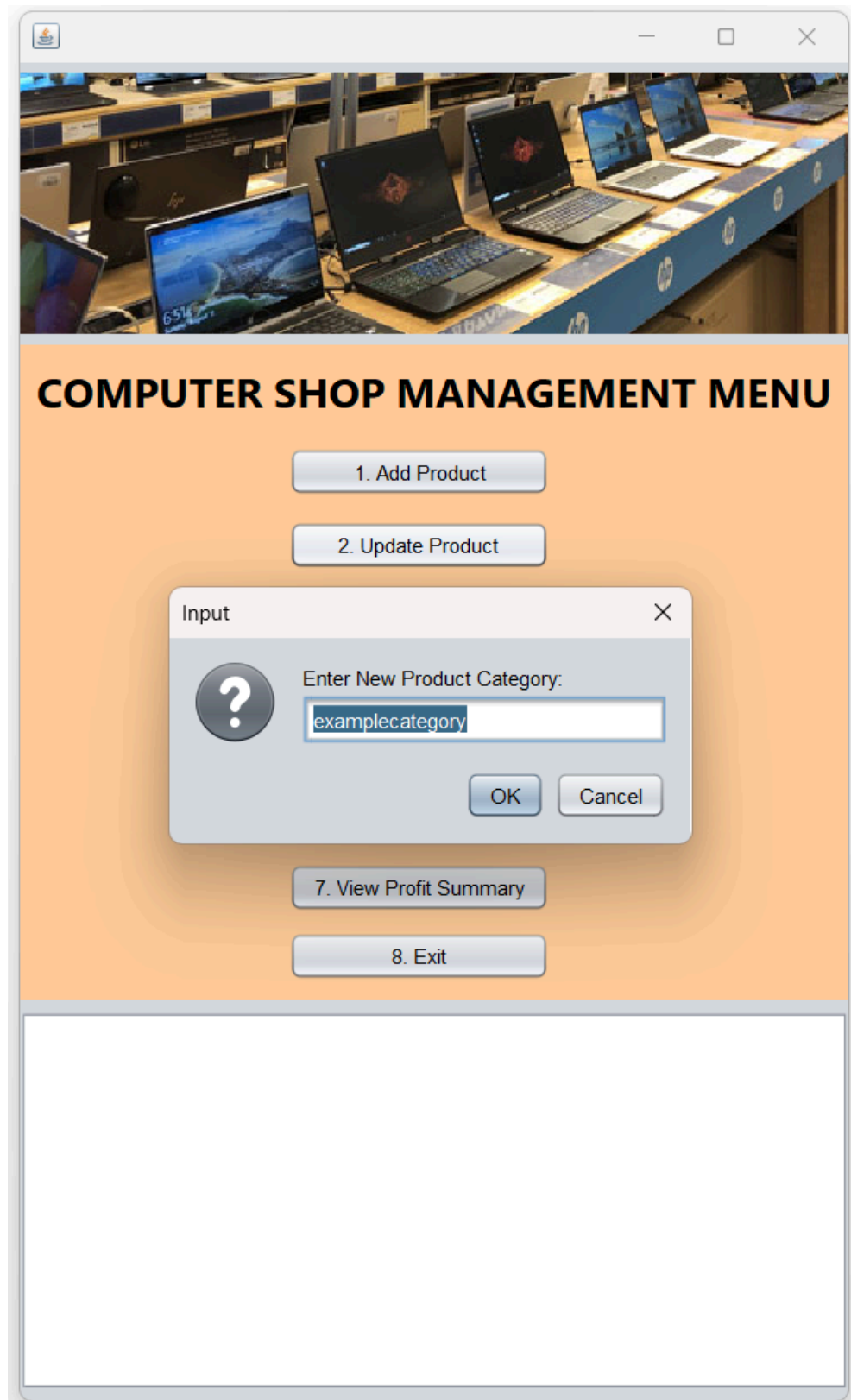


Image 3.4 : Enter New Product Category (Unchanged)

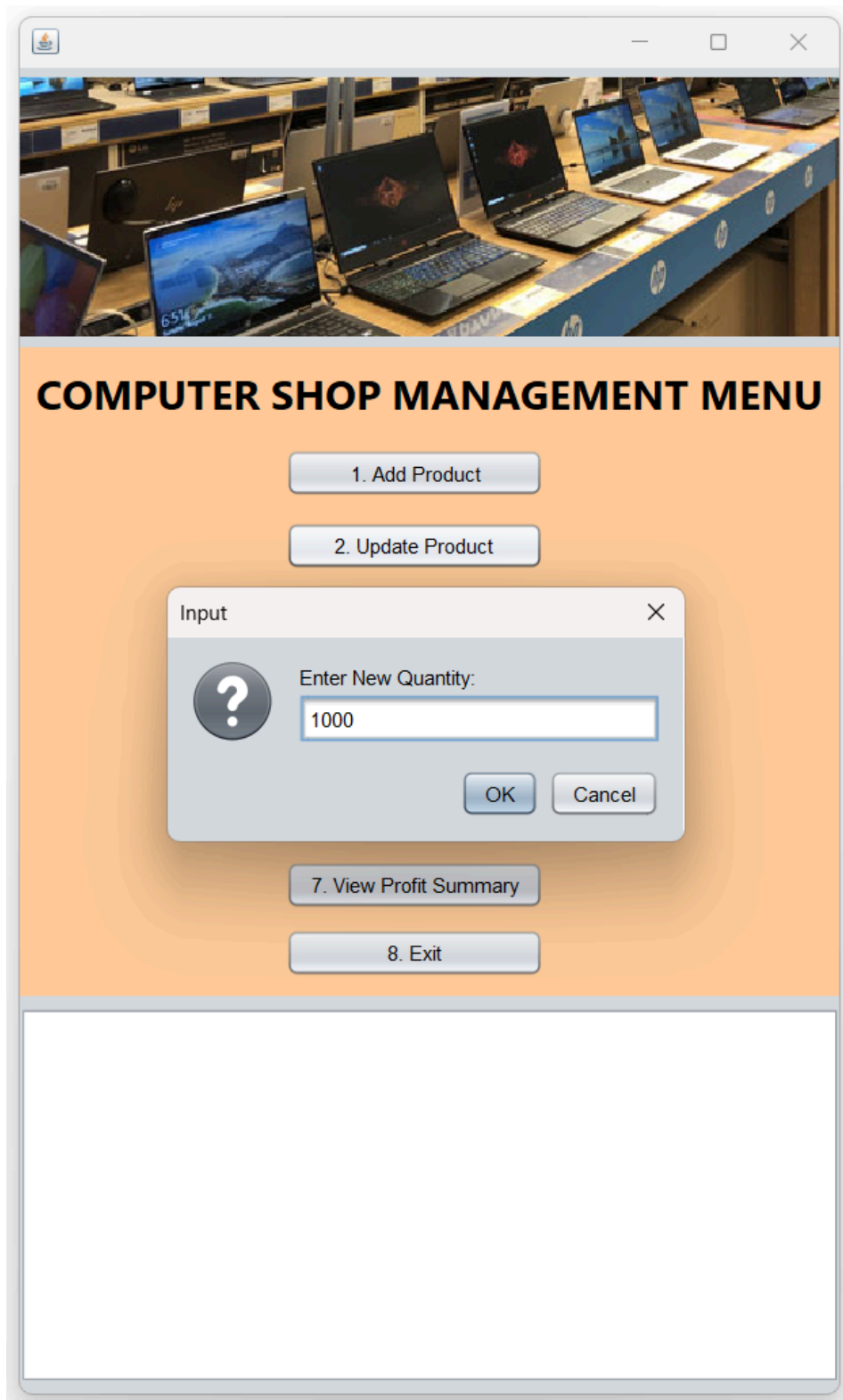


Image 3.5 : Enter New Quantity (Changed)

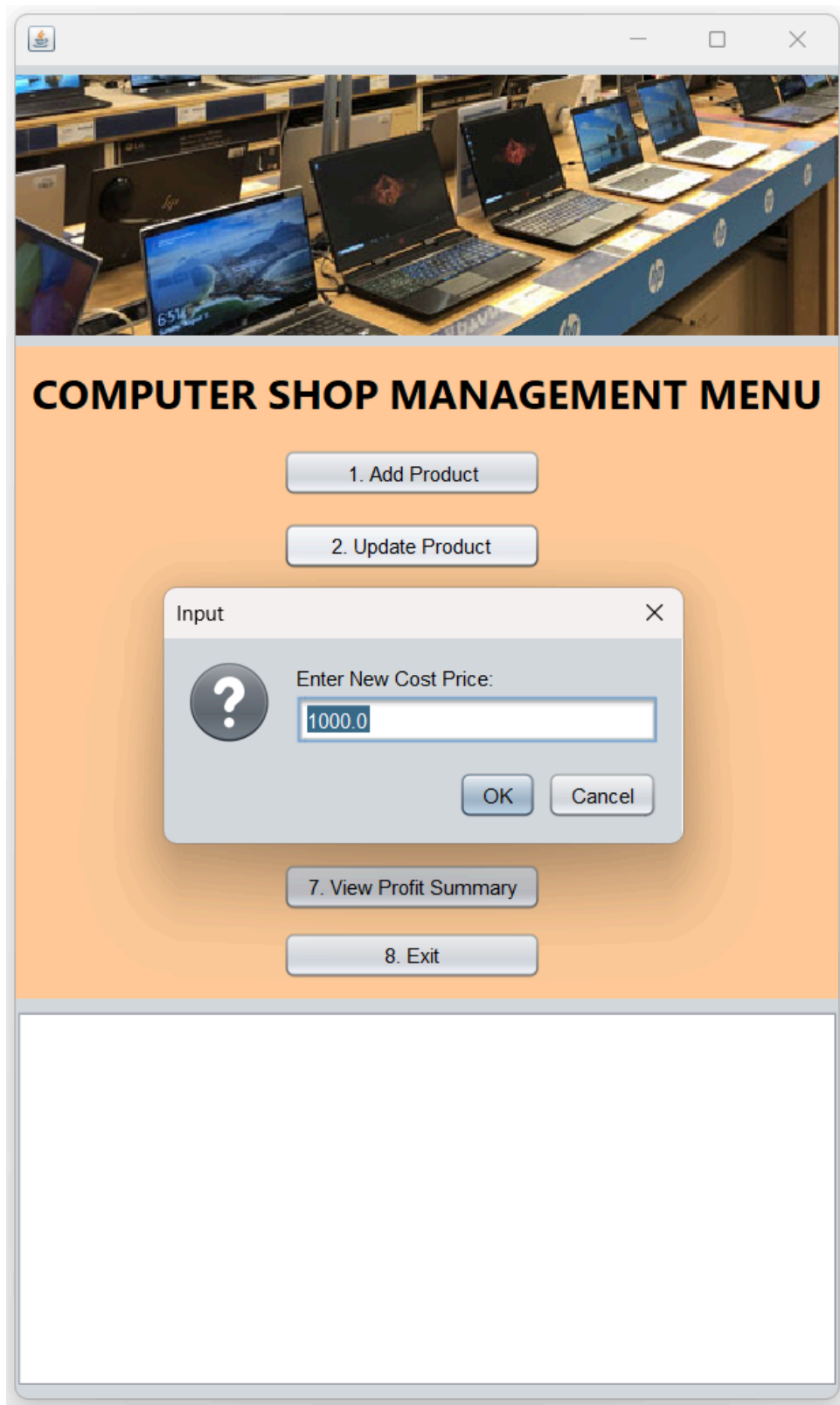


Image 3.6 : Enter New Cost Price (Unchanged)

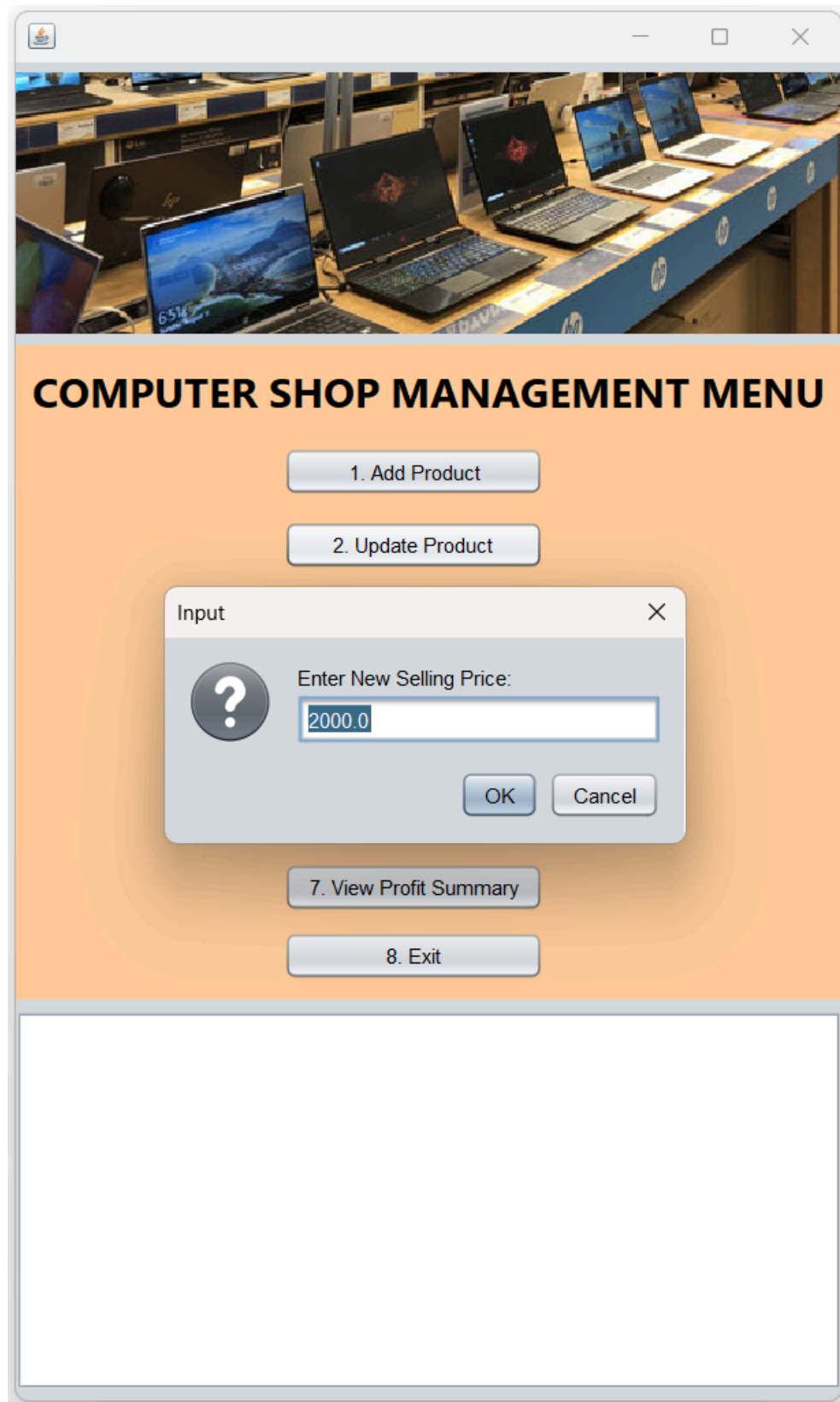


Image 3.7 : Enter New Selling Price (Unchanged)

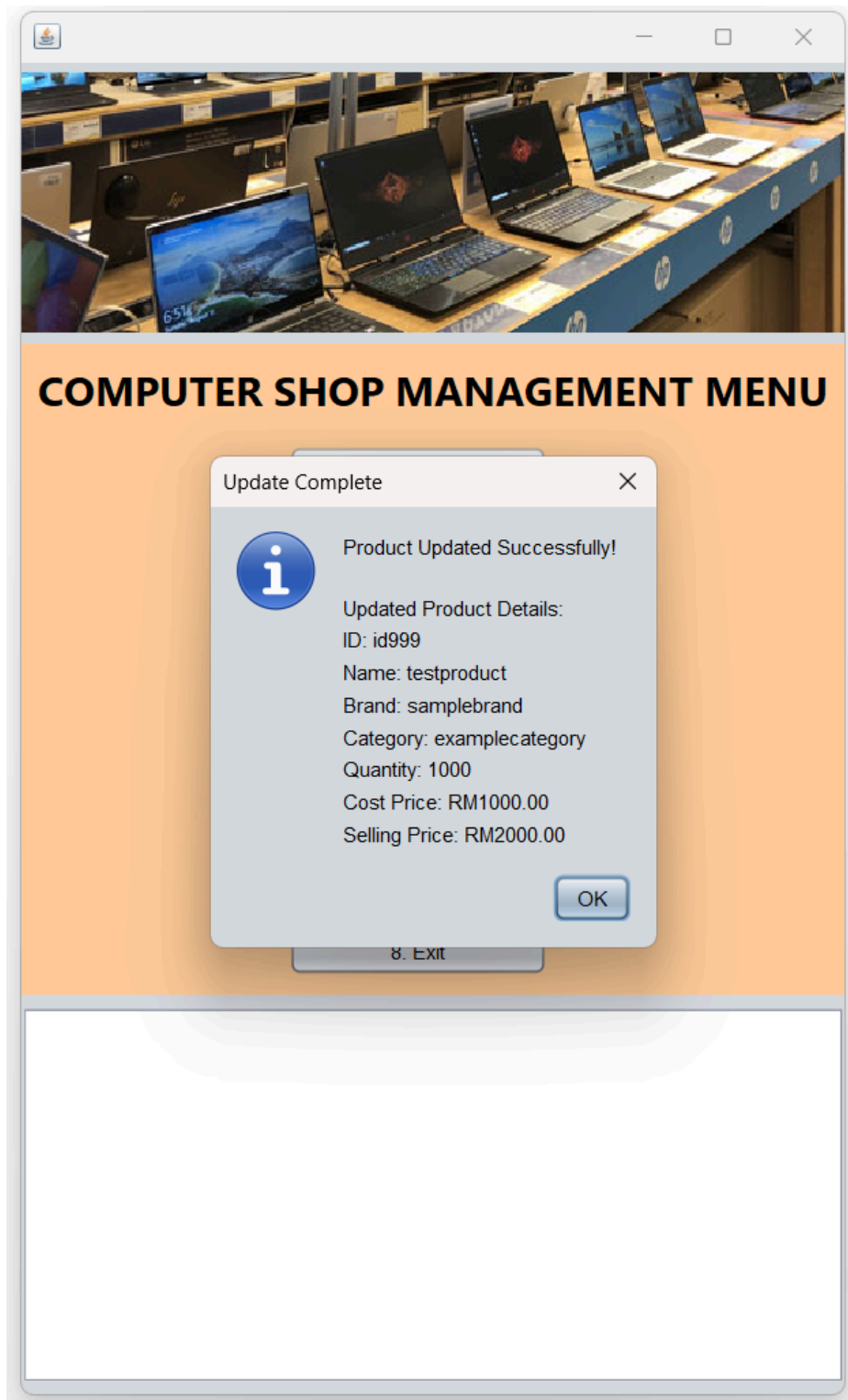


Image 3.8 : Product Updated Successfully

7.4. Delete Product Button

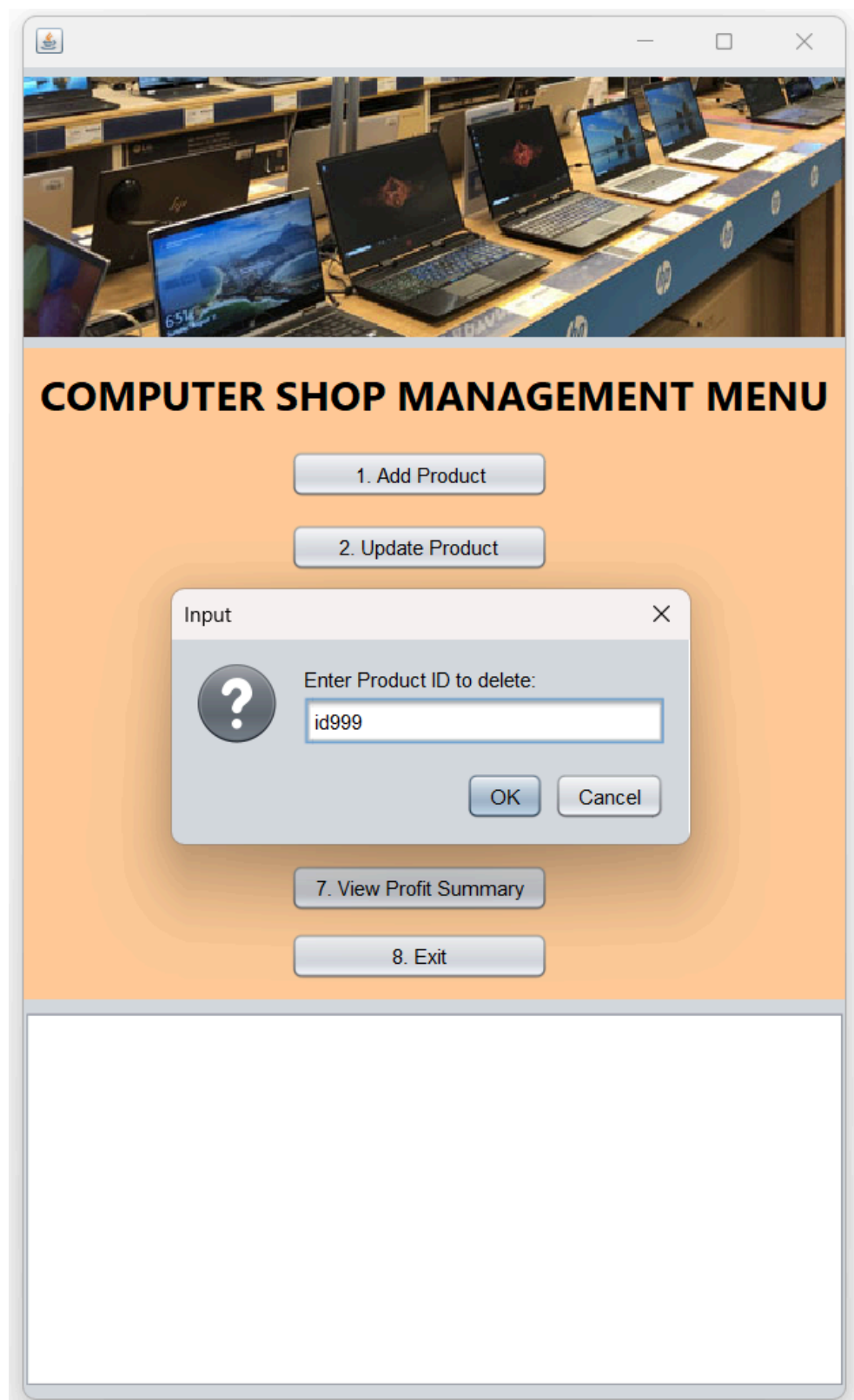


Image 4.0 : Enter Product ID to delete

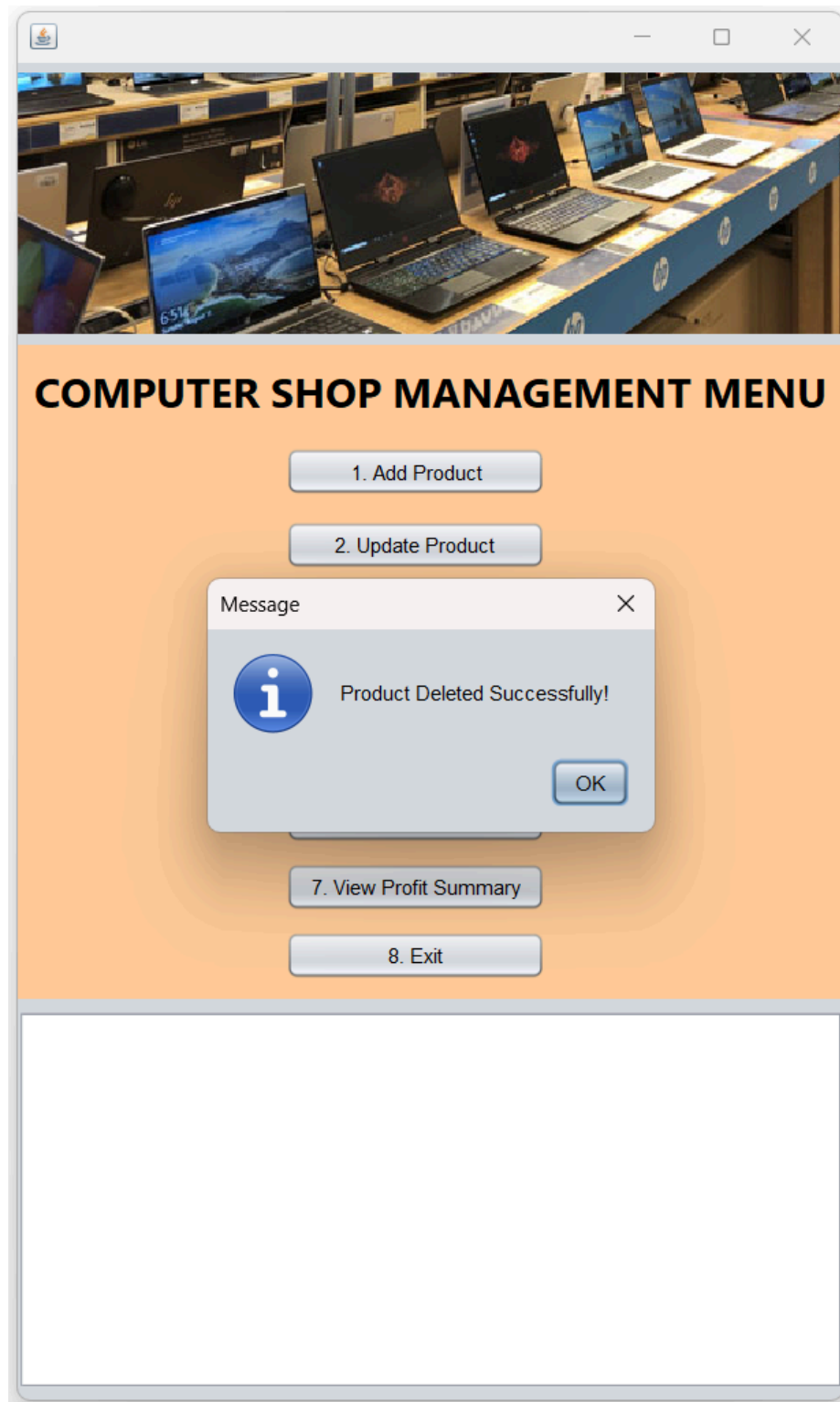


Image 4.1 : Product Deleted Successfully

7.5. View Products Button

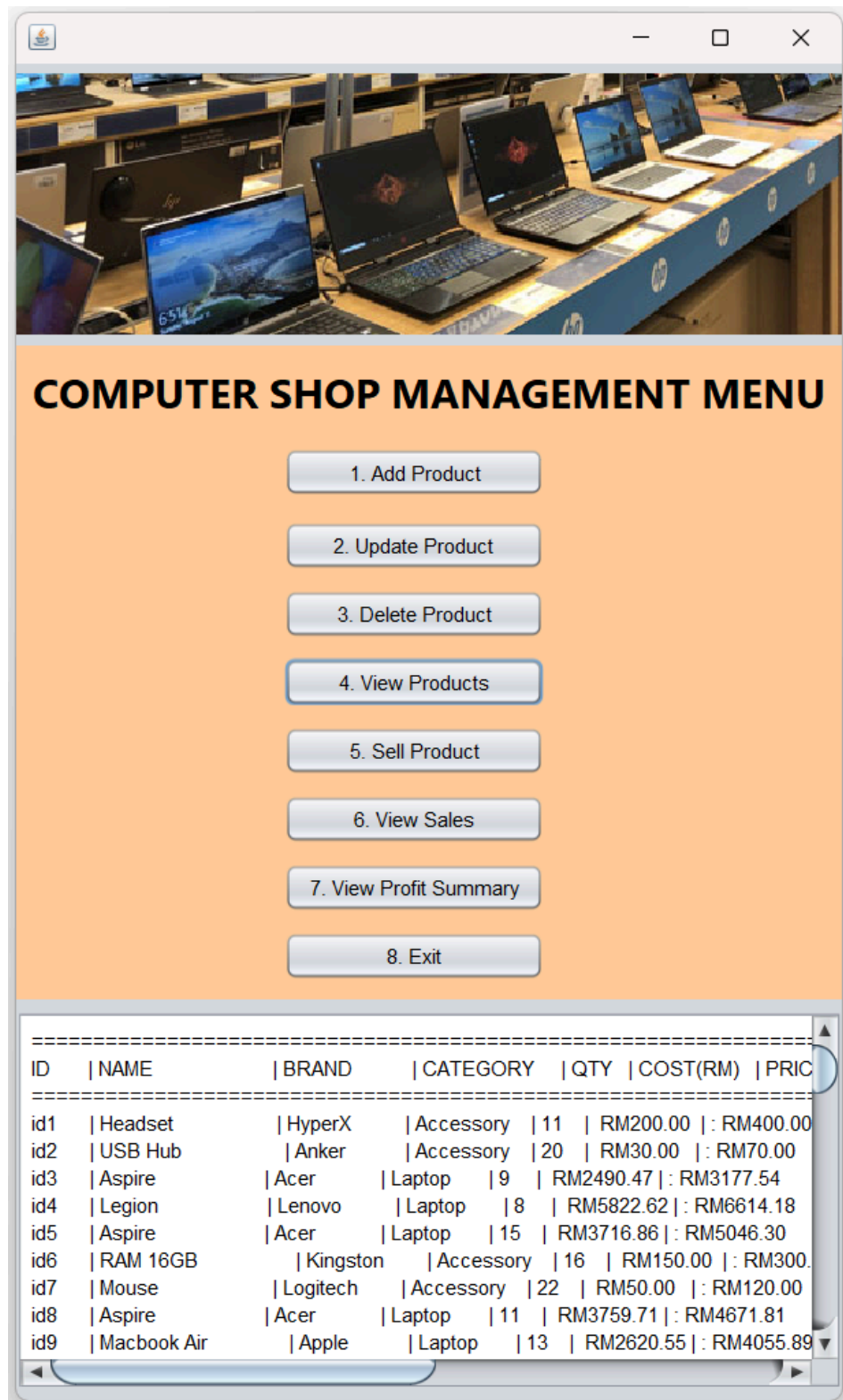
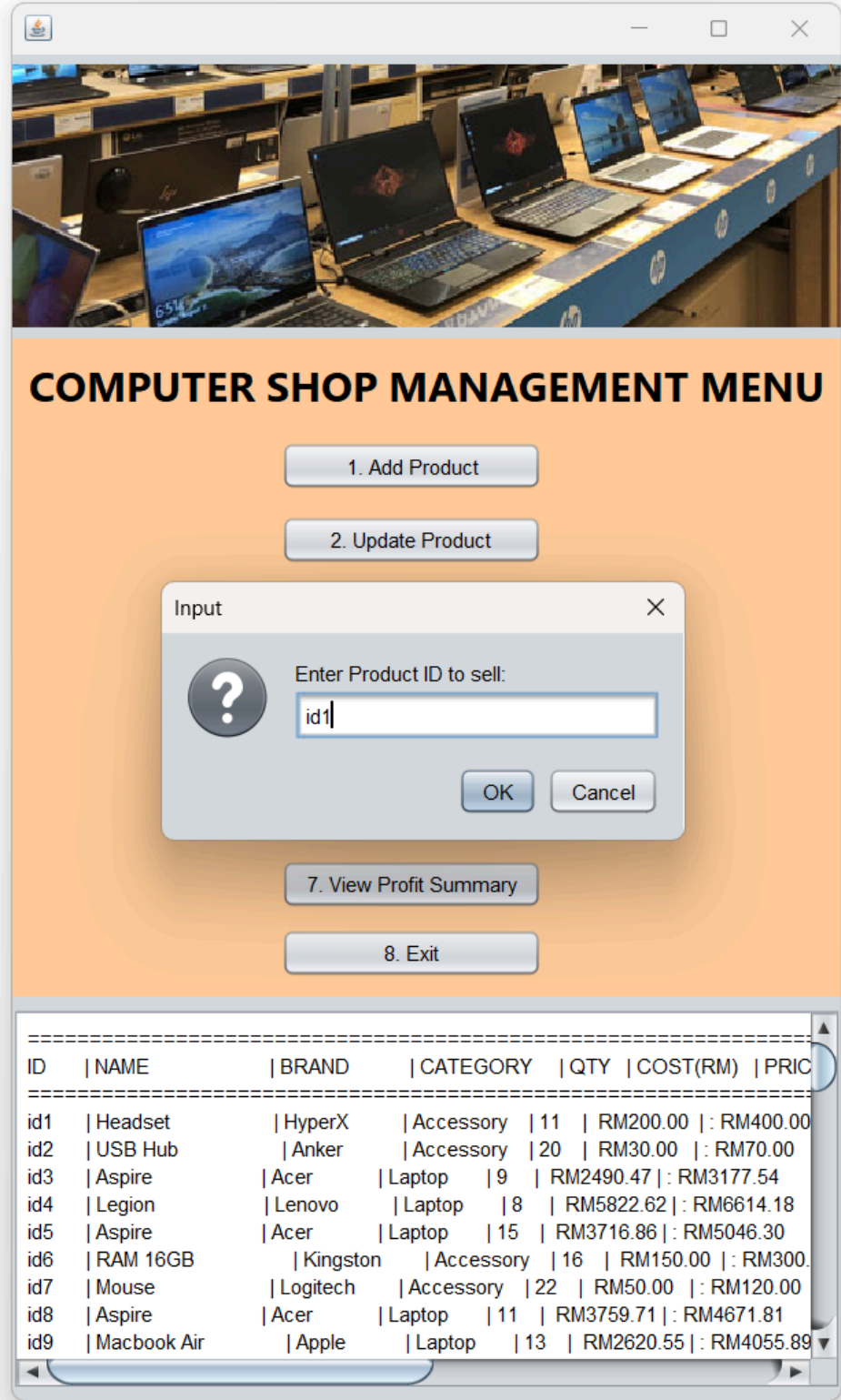


Image 5.0 : View Products

7.6. Sell Product Button



COMPUTER SHOP MANAGEMENT MENU

1. Add Product

2. Update Product

Input

Enter Product ID to sell:

id1

OK Cancel

7. View Profit Summary

8. Exit

ID	NAME	BRAND	CATEGORY	QTY	COST(RM)	PRICE(RM)
id1	Headset	HyperX	Accessory	11	RM200.00	RM400.00
id2	USB Hub	Anker	Accessory	20	RM30.00	RM70.00
id3	Aspire	Acer	Laptop	9	RM2490.47	RM3177.54
id4	Legion	Lenovo	Laptop	8	RM5822.62	RM6614.18
id5	Aspire	Acer	Laptop	15	RM3716.86	RM5046.30
id6	RAM 16GB	Kingston	Accessory	16	RM150.00	RM300.00
id7	Mouse	Logitech	Accessory	22	RM50.00	RM120.00
id8	Aspire	Acer	Laptop	11	RM3759.71	RM4671.81
id9	Macbook Air	Apple	Laptop	13	RM2620.55	RM4055.89

Image 6.0 : Enter Product ID to sell

COMPUTER SHOP MANAGEMENT MENU

1. Add Product

2. Update Product

Input

Enter Quantity to Sell (Available: 11):

1

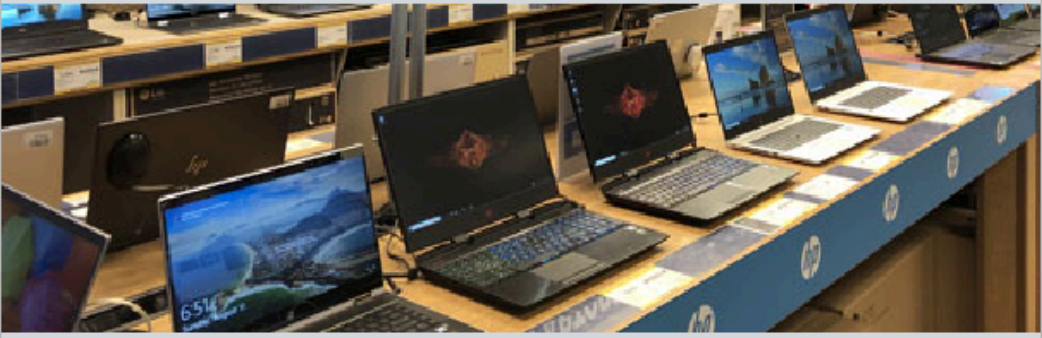
OK Cancel

7. View Profit Summary

8. Exit

ID	NAME	BRAND	CATEGORY	QTY	COST(RM)	PRICE
id1	Headset	HyperX	Accessory	11	RM200.00	RM400.00
id2	USB Hub	Anker	Accessory	20	RM30.00	RM70.00
id3	Aspire	Acer	Laptop	9	RM2490.47	RM3177.54
id4	Legion	Lenovo	Laptop	8	RM5822.62	RM6614.18
id5	Aspire	Acer	Laptop	15	RM3716.86	RM5046.30
id6	RAM 16GB	Kingston	Accessory	16	RM150.00	RM300.00
id7	Mouse	Logitech	Accessory	22	RM50.00	RM120.00
id8	Aspire	Acer	Laptop	11	RM3759.71	RM4671.81
id9	Macbook Air	Apple	Laptop	13	RM2620.55	RM4055.89

Image 6.1 : Enter Quantity to sell (Available Quantity)



COMPUTER SHOP MANAGEMENT MENU

1. Add Product

2. Update Product

7. View Profit Summary

8. Exit

Input

?

Enter Discount Amount (RM):

15

OKCancel

ID	NAME	BRAND	CATEGORY	QTY	COST(RM)	PRICE
id1	Headset	HyperX	Accessory	11	RM200.00	RM400.00
id2	USB Hub	Anker	Accessory	20	RM30.00	RM70.00
id3	Aspire	Acer	Laptop	9	RM2490.47	RM3177.54
id4	Legion	Lenovo	Laptop	8	RM5822.62	RM6614.18
id5	Aspire	Acer	Laptop	15	RM3716.86	RM5046.30
id6	RAM 16GB	Kingston	Accessory	16	RM150.00	RM300.00
id7	Mouse	Logitech	Accessory	22	RM50.00	RM120.00
id8	Aspire	Acer	Laptop	11	RM3759.71	RM4671.81
id9	Macbook Air	Apple	Laptop	13	RM2620.55	RM4055.89

Image 6.2 : Enter Discount Amount (RM)

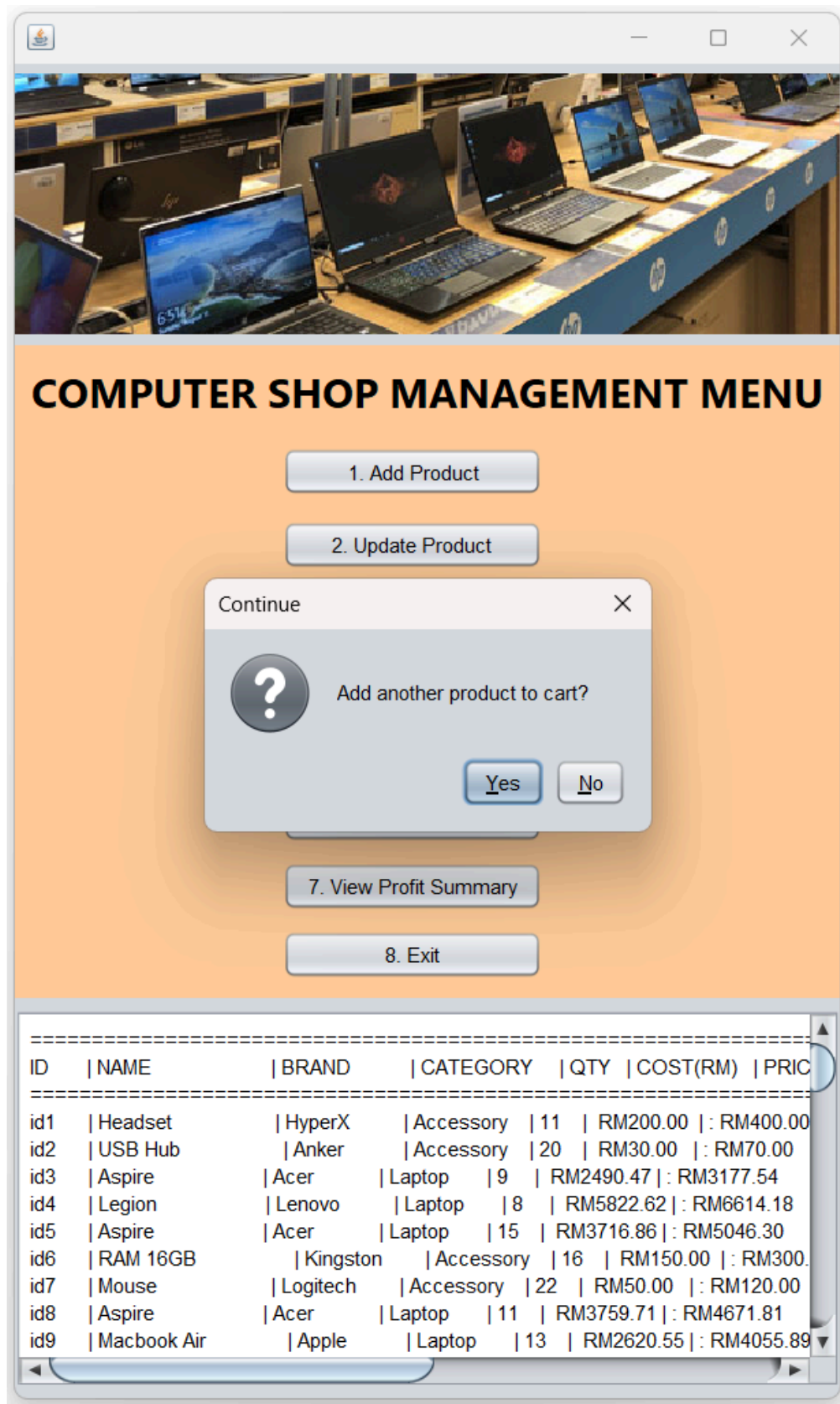
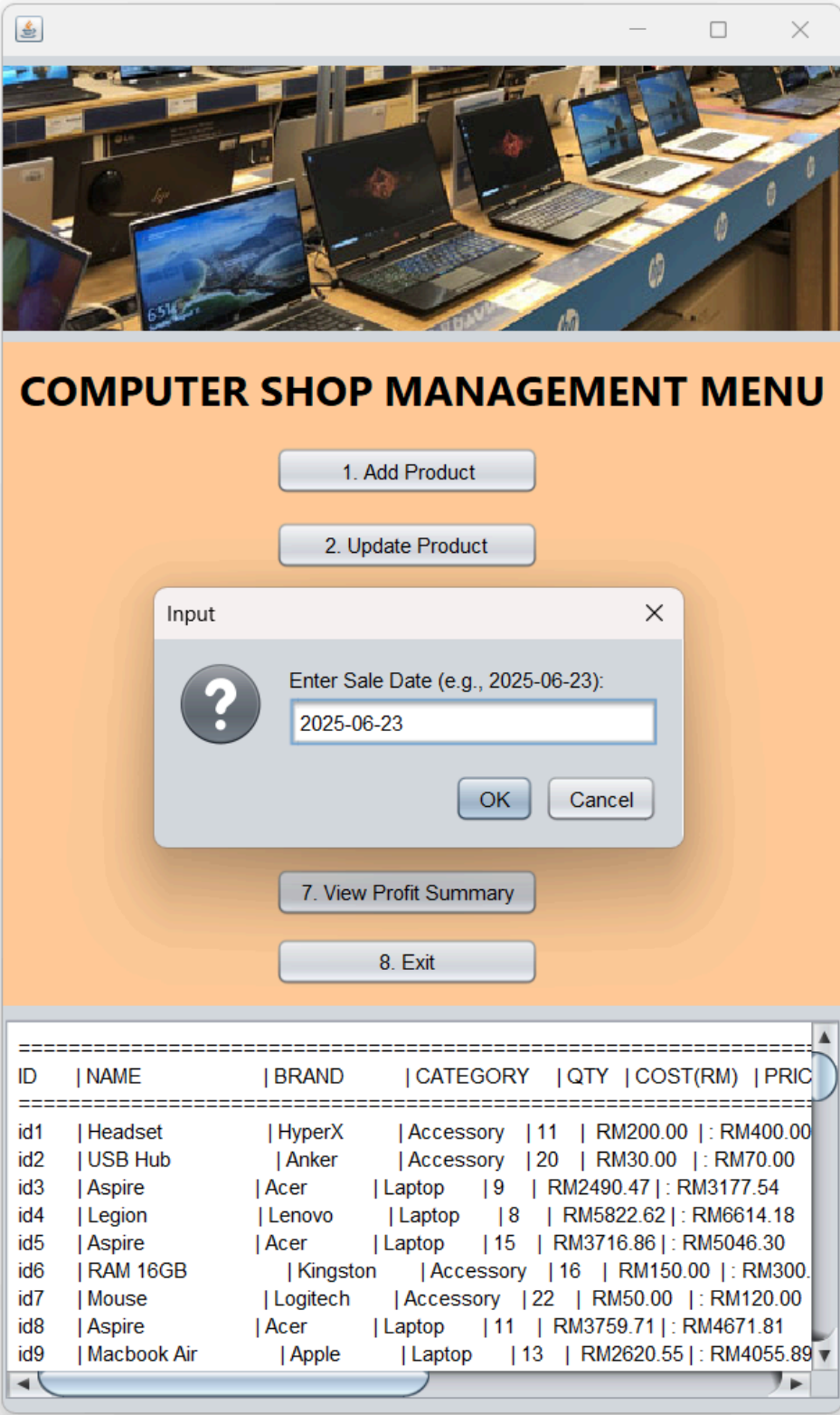


Image 6.3 : Add another product to cart (User clicks “No”)



COMPUTER SHOP MANAGEMENT MENU

1. Add Product

2. Update Product

Input

Enter Sale Date (e.g., 2025-06-23):

2025-06-23

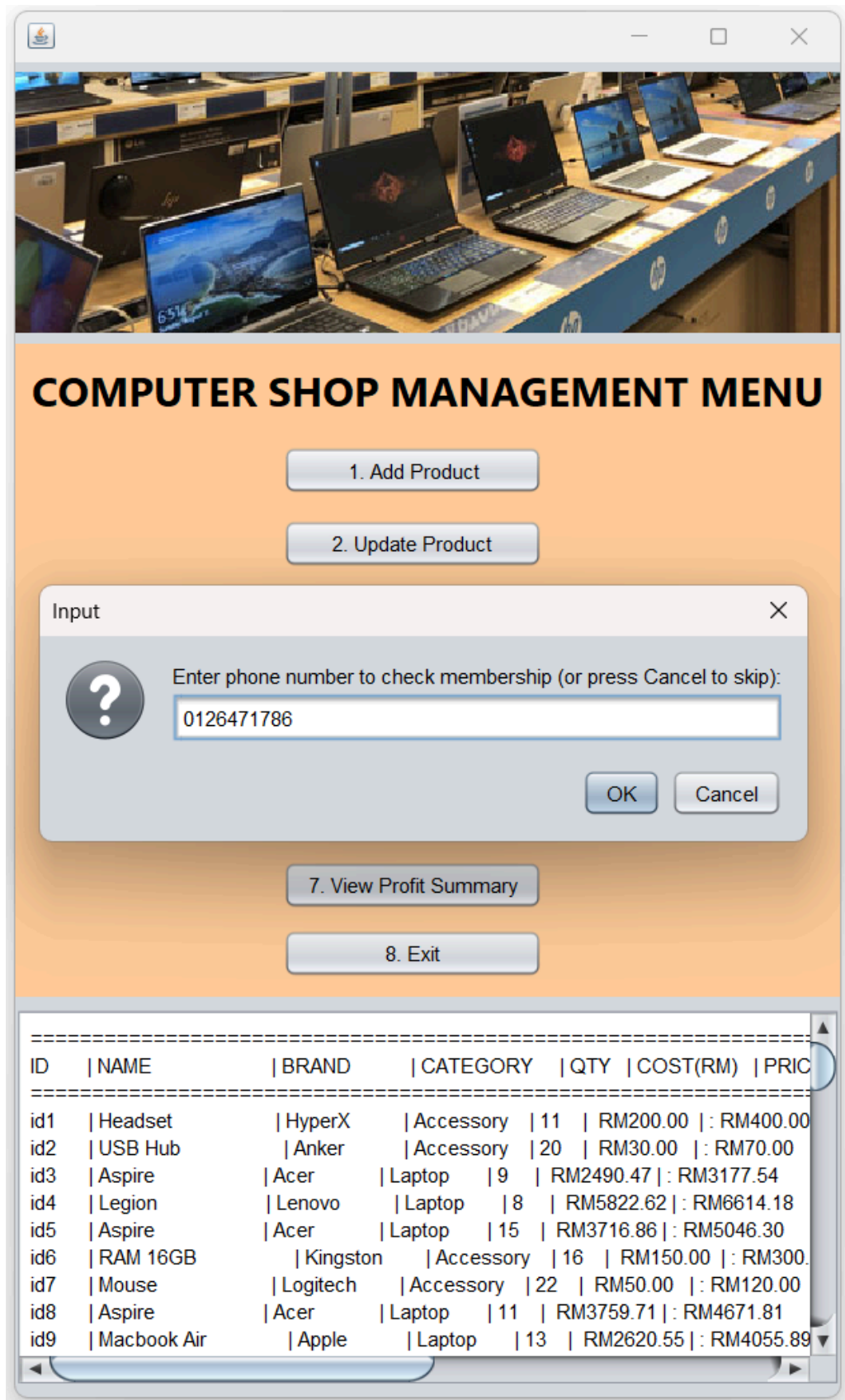
OK Cancel

7. View Profit Summary

8. Exit

ID	NAME	BRAND	CATEGORY	QTY	COST(RM)	PRICE
id1	Headset	HyperX	Accessory	11	RM200.00	RM400.00
id2	USB Hub	Anker	Accessory	20	RM30.00	RM70.00
id3	Aspire	Acer	Laptop	9	RM2490.47	RM3177.54
id4	Legion	Lenovo	Laptop	8	RM5822.62	RM6614.18
id5	Aspire	Acer	Laptop	15	RM3716.86	RM5046.30
id6	RAM 16GB	Kingston	Accessory	16	RM150.00	RM300.00
id7	Mouse	Logitech	Accessory	22	RM50.00	RM120.00
id8	Aspire	Acer	Laptop	11	RM3759.71	RM4671.81
id9	Macbook Air	Apple	Laptop	13	RM2620.55	RM4055.89

Image 6.4 : Enter Sale Date



COMPUTER SHOP MANAGEMENT MENU

1. Add Product

2. Update Product

Input

Enter phone number to check membership (or press Cancel to skip):

7. View Profit Summary

8. Exit

ID	NAME	BRAND	CATEGORY	QTY	COST(RM)	PRICE
id1	Headset	HyperX	Accessory	11	RM200.00	RM400.00
id2	USB Hub	Anker	Accessory	20	RM30.00	RM70.00
id3	Aspire	Acer	Laptop	9	RM2490.47	RM3177.54
id4	Legion	Lenovo	Laptop	8	RM5822.62	RM6614.18
id5	Aspire	Acer	Laptop	15	RM3716.86	RM5046.30
id6	RAM 16GB	Kingston	Accessory	16	RM150.00	RM300.00
id7	Mouse	Logitech	Accessory	22	RM50.00	RM120.00
id8	Aspire	Acer	Laptop	11	RM3759.71	RM4671.81
id9	Macbook Air	Apple	Laptop	13	RM2620.55	RM4055.89

Image 6.5 : Check Membership

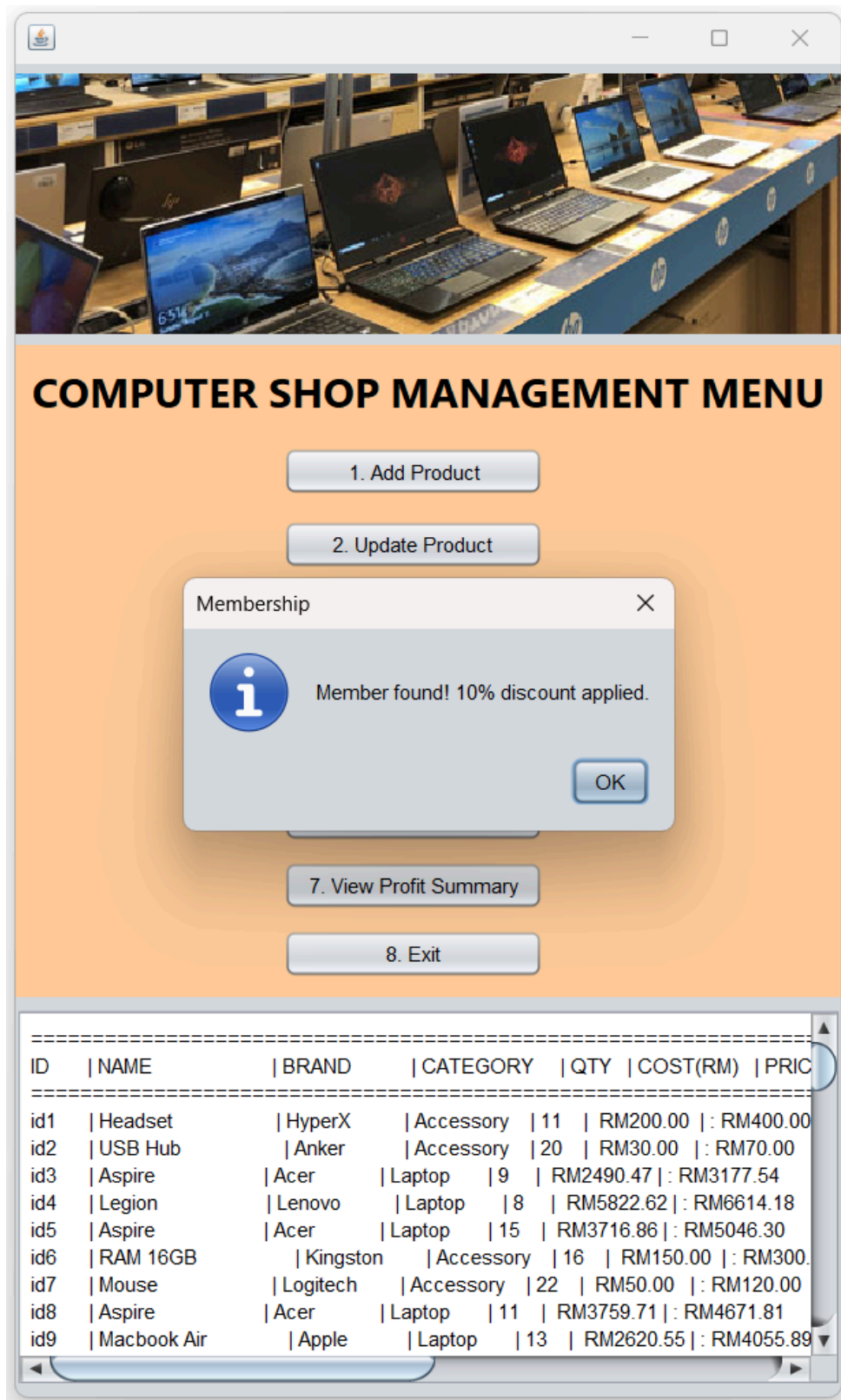


Image 6.6 : Member found and 10% discount applied

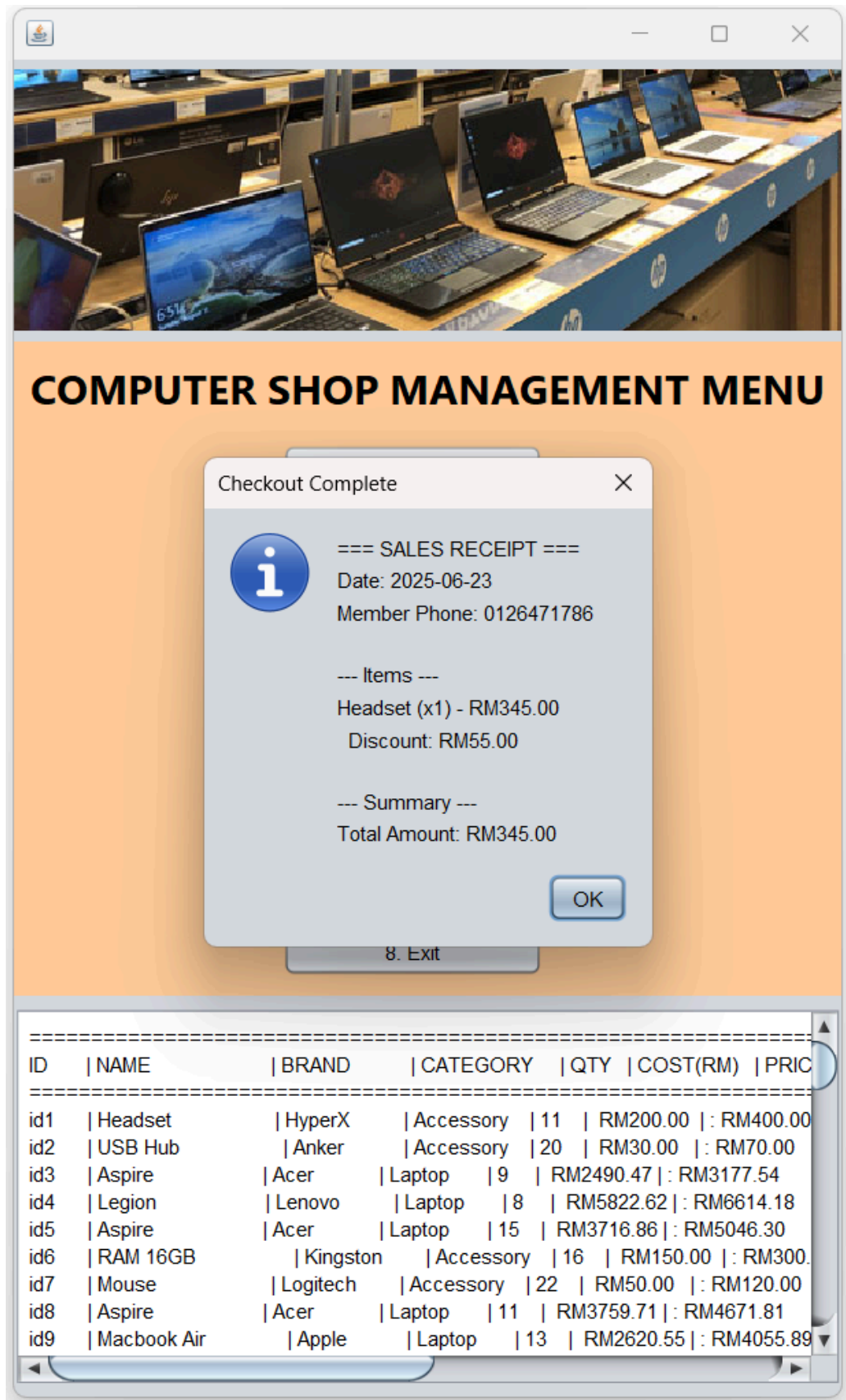


Image 6.7 : Checkout Complete

7.7. View Sales Button



Image 7.0 : View Sales

7.8. View Profit Summary Button

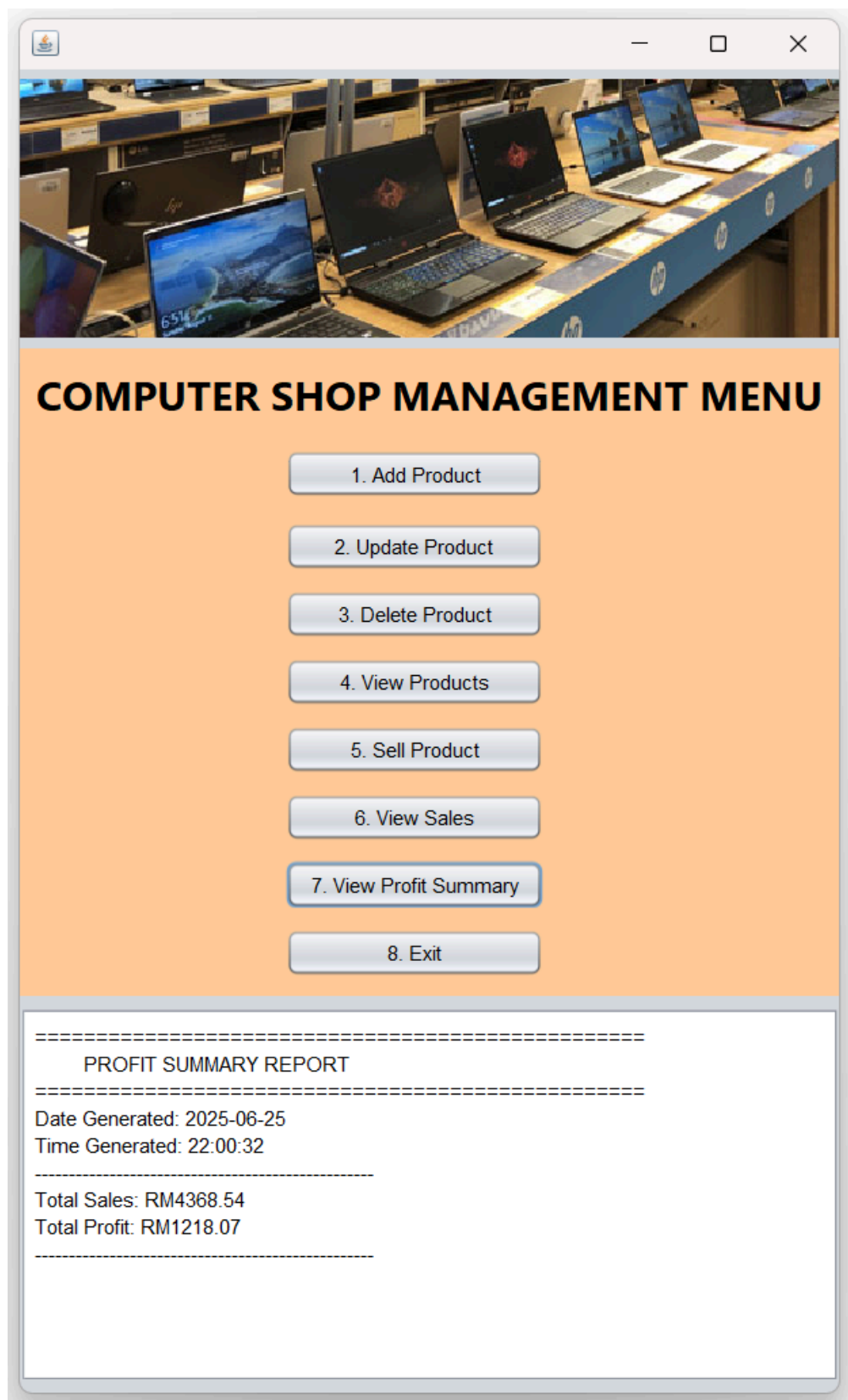
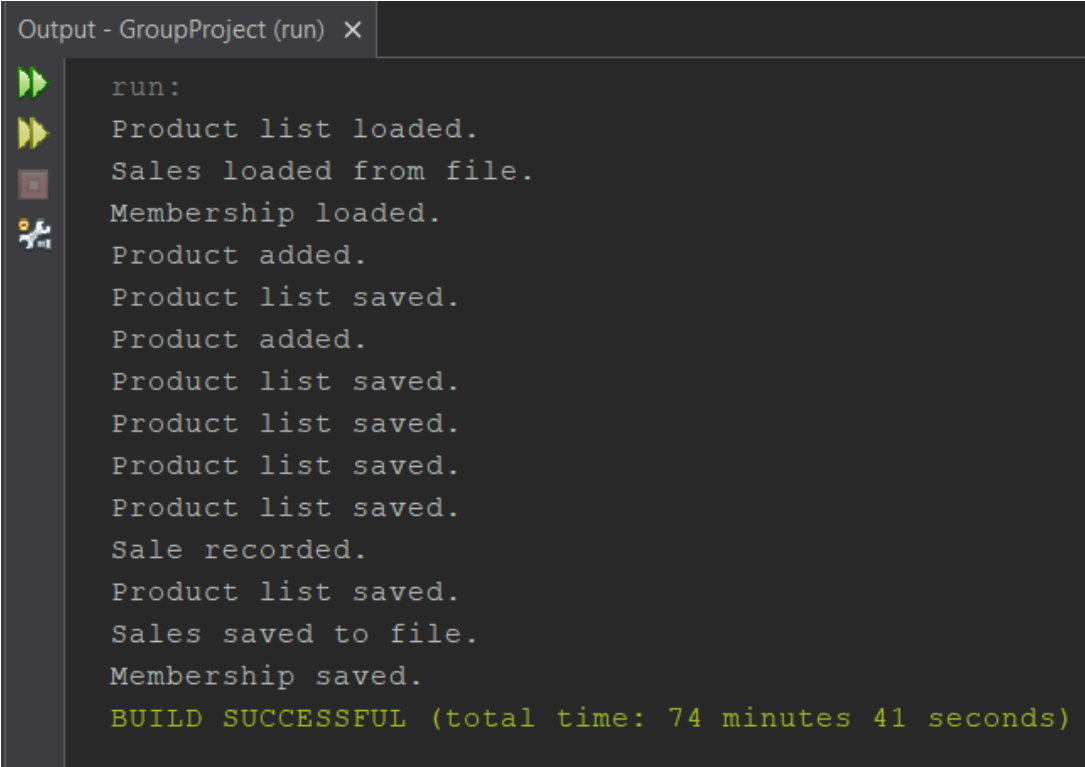


Image 8.0 : View Profit Summary

7.9. Exit Button



The screenshot shows an IDE's output window titled "Output - GroupProject (run) X". On the left side of the window, there is a vertical toolbar with four icons: a green play button, a yellow play button, a red stop button, and a white bug icon. The main area of the window displays the following text:

```
run:
Product list loaded.
Sales loaded from file.
Membership loaded.
Product added.
Product list saved.
Product added.
Product list saved.
Product list saved.
Product list saved.
Product list saved.
Sale recorded.
Product list saved.
Sales saved to file.
Membership saved.
BUILD SUCCESSFUL (total time: 74 minutes 41 seconds)
```

Image 9.0 : Exits Program

8. FUNCTION DESCRIPTIONS

8.1. Function 1: Add Product

8.1.1. Purpose

This function allows users to register new computer products in the inventory system.

8.1.2. GUI Operation

- Access: Click "1. Add Product" button on the main menu
- Data Entry: System prompts for the following information through sequential dialogs:
 - ◆ Product ID: Unique identifier for the product
 - ◆ Product Name: Descriptive name of the computer product
 - ◆ Brand: Manufacturer or brand name
 - ◆ Category: Product category classification
 - ◆ Quantity: Initial stock quantity
 - ◆ Cost Price: Purchase cost of the product
 - ◆ Selling Price: Retail price for customers
- Validation: System validates all input data for completeness and format
- Confirmation: Success message displayed upon successful addition
- Auto-Save: Product data is automatically saved to the file system

8.1.3. Console Operation

- Menu Selection: Enter "1" from the main menu

- Guided Input: System prompts for each field sequentially
- Multiple Entries: Option to add multiple products in a single session
- Validation: Real-time input validation and error handling
- Batch Processing: Ability to add multiple products before returning to main menu

8.1.4. Business Logic

- Unique product ID enforcement
- Automatic data persistence
- Input validation for numeric fields
- Integration with inventory management system

8.2. Function 2: Update Product

8.2.1. Purpose

Enables modification of existing product information including pricing and specifications.

8.2.2. GUI Operation

- Access: Click "2. Update Product" button
- Product Selection: Enter Product ID to identify the product for update
- Field Updates: System prompts for new values:
 - ◆ New Product Name: Updated product name
 - ◆ New Cost Price: Revised purchase cost
 - ◆ New Selling Price: Updated retail price
- Validation: System verifies product existence and validates new data

- Confirmation: Success message confirms the update
- Auto-Save: Changes are automatically saved to the file system

8.2.3. Console Operation

- Menu Selection: Enter "2" from the main menu
- Product Identification: Enter the Product ID to locate the item
- Field-by-Field Update: System guides through each updatable field
- Confirmation: System confirms changes before applying updates
- Error Handling: Clear error messages for invalid IDs or data

8.2.4. Business Logic

- Product existence verification
- Selective field updates (only name, cost price, and selling price)
- Data integrity maintenance
- Automatic file synchronization

8.3. Function 3: Delete Product

8.3.1. Purpose

Removes products from the inventory system when they are discontinued or obsolete.

8.3.2. GUI Operation

- Access: Click "3. Delete Product" button
- Product Selection: Enter Product ID for the item to be deleted
- Verification: System locates and identifies the product
- Confirmation: Success or failure message based on operation

result

→ Auto-Save: Inventory is automatically updated and saved

8.3.3. Console Operation

→ Menu Selection: Enter "3" from the main menu

→ Product Identification: Enter Product ID for deletion

→ Confirmation Process: System may require confirmation for critical deletions

→ Result Feedback: Clear indication of successful deletion or error messages

8.3.4. Business Logic

→ Product existence verification before deletion

→ Complete removal from inventory system

→ Data integrity maintenance

→ Automatic file system updates

8.4. Function 4: View Products

8.4.1. Purpose

Displays comprehensive product catalog with all available inventory information.

8.4.2. GUI Operation

→ Access: Click "4. View Products" button

→ Display Window: Product list appears in a popup dialog box

→ Information Display: Each product shows:

◆ Product ID and Name

◆ Brand and Category

◆ Current Quantity in stock

◆ Cost Price and Selling Price

- Scrollable Interface: Large inventories can be scrolled through
- Empty State: Appropriate message when no products are available

8.4.3. Console Operation

- Menu Selection: Enter "4" from the main menu
- Formatted Display: Products listed in organized, readable format
- Complete Listing: All products displayed with full details
- Navigation: Automatic return to main menu after viewing

8.4.4. Business Logic

- Real-time inventory display
- Formatted output for readability
- Complete product information presentation
- Dynamic content based on current inventory

8.5. Function 5: Sell Product

8.5.1. Purpose

Processes customer transactions with advanced cart functionality, discount application, and membership benefits.

8.5.2. GUI Operation

The GUI implementation of the "Sell Product" functionality provides a comprehensive cart-based transaction processing system with the following workflow:

→ **Cart Initialization and Product Selection Loop:**

- ◆ Cart Reset: System clears any previous cart items to

start fresh

- ◆ Product Display: Shows complete product list in a popup dialog for easy selection
- ◆ Product Entry: User enters Product ID through input dialog
- ◆ Product Validation: System verifies product exists and displays error message if not found
- ◆ Stock Verification: Checks product availability and prevents sale of out-of-stock items

→ Quantity and Discount Processing:

- ◆ Quantity Input: System prompts for quantity with current stock level displayed (e.g., "Available: 15")
- ◆ Quantity Validation: Ensures requested quantity is between 1 and available stock
- ◆ Discount Entry: User enters manual discount amount in RM
- ◆ Input Validation: System validates all numeric inputs with error handling for invalid entries

→ Cart Management:

- ◆ Item Addition: Valid items are added to cart with all details (ID, name, quantity, unit price, discount)
- ◆ Continuation Option: System asks if user wants to add more products to cart
- ◆ Multiple Items: Supports adding multiple different products in single transaction

- ◆ Cart Validation: Prevents checkout if cart is empty

→ **Transaction Details:**

- ◆ Date Entry: System prompts for sale date in standard format (e.g., "2025-06-23")
- ◆ Date Validation: User input required before proceeding to membership check

→ **Membership Processing:**

- ◆ Phone Number Entry: Optional phone number input for membership verification
- ◆ Membership Check: System automatically verifies existing membership status
- ◆ New Member Registration: For non-members, system offers registration option with name input
- ◆ Member Benefits: Automatic 10% discount application for verified members
- ◆ Registration Confirmation: Success message displayed for new member registrations

→ **Transaction Processing and Inventory Updates:**

- ◆ Stock Reduction: Automatic inventory quantity reduction for each sold item
- ◆ Sales Recording: Each cart item recorded in sales system with complete details
- ◆ Discount Calculation: System calculates total discounts including membership benefits
- ◆ Total Calculation: Final amount computed after all

discounts applied

→ **Receipt Generation and Display:**

- ◆ Formatted Receipt: Professional receipt display with header and date
- ◆ Member Information: Member phone number included if applicable
- ◆ Itemized Listing: Each product listed with quantity and calculated total
- ◆ Discount Breakdown: Individual item discounts clearly displayed
- ◆ Summary Section: Total amount prominently shown
- ◆ Popup Display: Complete receipt shown in information dialog

→ **Data Persistence:**

- ◆ Automatic Saving: All changes saved to file system immediately after transaction
- ◆ Inventory Sync: Product quantities updated in storage
- ◆ Sales Recording: Transaction details saved to sales file
- ◆ Member Database: New memberships saved to member file

→ **Error Handling and User Experience:**

- ◆ Input Validation: Comprehensive validation for all user inputs
- ◆ Error Messages: Clear, specific error messages for different scenarios

- ◆ Cancellation Support: User can cancel at any stage using Cancel button
- ◆ Confirmation Dialogs: Clear success messages and confirmations
- ◆ Professional Interface: Clean, intuitive dialog-based interface

→ **Advanced Features:**

- ◆ Multi-Product Transactions: Full cart functionality supporting multiple items
- ◆ Dynamic Pricing: Real-time price calculations with discount applications
- ◆ Member Integration: Seamless integration with membership system
- ◆ Stock Management: Real-time inventory tracking and updates
- ◆ Professional Receipts: Comprehensive transaction documentation

8.5.3. Console Operation

This is the most sophisticated function in the system, featuring:

→ **Cart Initialization:**

- ◆ Creates empty shopping cart for the transaction
- ◆ Initializes discount and total tracking variables

→ **Product Selection Loop:**

- ◆ Product Entry: Enter Product ID for items to add to cart
- ◆ Inventory Check: System verifies product existence and

availability

- ◆ Quantity Validation: Ensures requested quantity doesn't exceed stock

- ◆ Stock Verification: Prevents out-of-stock sales

→ **Pricing and Discounts:**

- ◆ Unit Price Calculation: Retrieves current selling price

- ◆ Subtotal Calculation: Multiplies unit price by quantity

- ◆ Tiered Discount Application:

 - 8% discount for subtotals over RM 10,000

 - 5% discount for subtotals over RM 5,000

- ◆ Cart Addition: Adds item with calculated discounts to cart

→ **Membership Processing:**

- ◆ Membership Eligibility: Available for purchases over RM 5,000

- ◆ Member Verification: Check existing membership by phone number

- ◆ New Member Registration: Option to register new members

- ◆ Additional Discount: 2% extra discount for members

- ◆ Benefits Application: Automatic application of member benefits

→ **Transaction Summary:**

- ◆ Cart Display: Shows all items with quantities and individual totals

- ◆ Discount Breakdown: Details all applied discounts and savings

- ◆ Final Total: Displays amount to be paid after all discounts

→ **Payment Processing:**

- ◆ Confirmation: User confirms payment intent

- ◆ Date Entry: Records transaction date

- ◆ Inventory Update: Reduces stock quantities for sold items

- ◆ Sales Recording: Logs all transaction details

- ◆ Data Persistence: Saves all changes to file system

→ **7.5.4 Business Logic**

- ◆ Multi-item Cart System: Supports multiple products in single transaction

- ◆ Complex Discount Logic: Tiered discounts plus membership benefits

- ◆ Inventory Management: Real-time stock updates

- ◆ Customer Relationship: Membership management and benefits

- ◆ Financial Tracking: Comprehensive sales recording

8.6. Function 6: View Sales

8.6.1. Purpose

Displays comprehensive sales history and transaction records for business analysis.

8.6.2. GUI Operation

- Access: Click "6. View Sales" button
- Sales Display: Complete sales history appears in popup dialog
- Transaction Details: Each sale shows:
 - ◆ Product ID and quantity sold
 - ◆ Unit price and applied discounts
 - ◆ Final total and transaction date
- Scrollable View: Easy navigation through sales history
- Empty State: Appropriate message when no sales are recorded

8.6.3. Console Operation

- Menu Selection: Enter "6" from the main menu
- Formatted Report: Sales displayed in organized, readable format
- Complete History: All recorded transactions with full details
- Automatic Return: Returns to main menu after display

8.6.4. Business Logic

- Comprehensive Reporting: All sales transactions included
- Detailed Information: Complete transaction data display
- Real-time Data: Current sales information from file system
- Financial Analysis: Foundation for business decision-making

8.7. Function 7: View Profit Summary

8.7.1. Purpose

Provides financial analysis and profit calculations for business performance evaluation.

8.7.2. GUI Operation

- Access: Click "7. View Profit Summary" button

- Profit Display: Total profit figure appears in popup dialog
- Formatted Output: Professional display of financial information
- Quick Analysis: Immediate access to key financial metrics

8.7.3. Console Operation

- Menu Selection: Enter "7" from the main menu
- Profit Calculation: System calculates total profit from all sales
- Formatted Display: Professional presentation of financial data
- Business Insights: Key performance indicators for decision-making

8.7.4. Business Logic

- Automatic Calculation: Sums all final totals from sales records
- Real-time Analysis: Current profit based on recorded transactions
- Financial Accuracy: Precise calculations for business planning
- Performance Metrics: Essential data for business evaluation

8.8. Function 8: Exit

8.8.1. Purpose

Safely terminates the application while ensuring all data is properly saved.

8.8.2. GUI Operation

- Access: Click "8. Exit" button
- Data Preservation: System saves all membership data
- Clean Shutdown: Proper application termination
- System Exit: Complete closure of application

8.8.3. Console Operation

- Menu Selection: Enter "8" from the main menu
- Farewell Message: System displays goodbye message
- Data Saving: All data automatically saved before exit
- Safe Termination: Proper resource cleanup and application closure

8.8.4. Business Logic

- Data Integrity: Ensures all data is saved before exit
- Clean Shutdown: Proper resource management
- User Confirmation: Clear indication of system shutdown
- Data Persistence: Guarantees data availability for next session

9. TECHNICAL ARCHITECTURE

9.1. 8.1 Object-Oriented Design

9.1.1. Class Structure

- **Product (Abstract Base Class):** Defines common product attributes and methods
- **Computer (Concrete Class):** Extends Product for computer-specific implementations
- **Cart_item:** Represents individual items in shopping cart with discount calculations
- **Sales:** Encapsulates individual sales transaction data
- **Manager Classes:** Handle business logic and data operations

9.1.2. Design Patterns

- **Inheritance:** Product hierarchy with Computer extending Product
- **Encapsulation:** Private fields with public getter/setter methods
- **Abstraction:** Abstract Product class defining common interface
- **Composition:** Manager classes containing collections of domain objects

9.2. Data Management

9.2.1. File-Based Storage

- **Text File Format:** Human-readable CSV-like format for data storage
- **Automatic Persistence:** Data saved after each significant operation

→ **Error Handling:** Robust file I/O with exception management

→ **Data Recovery:** System loads existing data on startup

9.2.2. Data Integrity

→ **Input Validation:** Comprehensive validation for all user inputs

→ **Error Handling:** Graceful handling of invalid data and system errors

→ **Backup Strategy:** Automatic data saving prevents data loss

→ **Consistency Checks:** Validation ensures data consistency across operations

10. DATA MANAGEMENT

10.1. Storage Structure

10.1.1. File Organization

- **Products File:** computers.txt - stores all product information
- **Sales File:** sales.txt - contains complete sales transaction history
- **Members File:** members.txt - maintains customer membership records
- **Directory Structure:** Organized folder structure for data files

10.1.2. Data Format

- **CSV Format:** Comma-separated values for easy parsing and human readability
- **Field Separation:** Clear delimiters for data integrity
- **Line-based Records:** Each record on separate line for easy processing
- **Standardized Format:** Consistent data structure across all files

10.2. Data Operations

10.2.1. CRUD Operations

- **Create:** Add new records to appropriate files
- **Read:** Load data from files into application memory
- **Update:** Modify existing records and save changes
- **Delete:** Remove records from files and memory

10.2.2. Data Synchronization

- **Automatic Saving:** Changes saved immediately after

operations

- **Load on Startup:** System loads all data when application starts
- **Real-time Updates:** In-memory data synchronized with file system
- **Error Recovery:** Robust error handling for data operations

11. CONCLUSION

11.1. System Benefits

The Computer Shop Management System provides significant advantages over manual operations:

11.1.1. Operational Efficiency

- **Time Savings:** Automated processes reduce transaction time by 60-70%
- **Error Reduction:** Automated calculations eliminate human calculation errors
- **Improved Productivity:** Staff can focus on customer service rather than manual calculations
- **Streamlined Operations:** Integrated system reduces operational complexity

11.1.2. Business Intelligence

- **Sales Analytics:** Comprehensive sales reporting enables data-driven decisions
- **Inventory Insights:** Real-time stock levels prevent stockouts and overstock situations
- **Customer Management:** Membership system provides customer retention benefits
- **Financial Clarity:** Automated profit calculations provide clear financial visibility

11.1.3. Customer Experience

- **Faster Service:** Reduced transaction time improves customer satisfaction

- **Accurate Pricing:** Automated calculations ensure pricing accuracy
- **Membership Benefits:** Transparent discount system encourages customer loyalty
- **Professional Service:** System-supported operations enhance service quality

11.2. System Scalability

The system is designed for growth and adaptation:

11.2.1. Technical Scalability

- **Modular Architecture:** Easy to add new features and functionality
- **Object-Oriented Design:** Facilitates system extension and modification
- **Flexible Data Structure:** Can accommodate additional product types and attributes
- **Interface Options:** Both GUI and console interfaces support different user preferences

11.2.2. Business Scalability

- **Multi-Product Support:** System can handle diverse computer product categories
- **Volume Handling:** Efficient data structures support growing inventory and sales volumes
- **User Accommodation:** System supports multiple user types and roles
- **Feature Expansion:** Architecture allows for additional

business functions

11.3. Future Enhancements

Potential system improvements include:

11.3.1. Technical Enhancements

- **Database Integration:** Migration from file-based to database storage
- **Network Capabilities:** Multi-user access and network deployment
- **Web Interface:** Browser-based access for remote management
- **Mobile Support:** Mobile applications for on-the-go management

11.3.2. Functional Enhancements

- **Advanced Reporting:** More sophisticated business analytics and reporting
- **Supplier Management:** Integration with supplier systems for automated ordering
- **Customer Analytics:** Detailed customer behavior analysis and segmentation
- **Integration Options:** Connection with accounting systems and e-commerce platforms

11.4. Final Assessment

The Computer Shop Management System successfully addresses the critical needs of computer retail operations by providing a comprehensive, efficient, and user-friendly solution. The system's dual interface approach, robust feature set, and solid technical foundation make it an ideal choice for

computer retailers seeking to modernize their operations and improve business performance.

- 11.5. The implementation demonstrates strong software engineering principles with its object-oriented design, comprehensive error handling, and user-centric interface design. The system's ability to handle complex business logic while maintaining simplicity for end users makes it a valuable tool for any computer retail business.
- 11.6.** Through automated inventory management, sophisticated sales processing, customer relationship management, and comprehensive reporting capabilities, this system transforms traditional manual operations into efficient, accurate, and insights-driven business processes. The result is improved operational efficiency, enhanced customer service, and better business decision-making capabilities that position computer retailers for success in today's competitive market.

12. SOURCE CODE

12.1. Cart_item.java :

```
/*
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-de
fault.txt to change this license
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */

public class Cart_item {

    private String productId;
    private String name;
    private int quantity;
    private double unitPrice;
    private double discountAmount;

    public Cart_item(String productId, String name, int
quantity, double unitPrice, double discountAmount) {
        this.productId = productId;
        this.name = name;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.discountAmount = discountAmount;
    }

    public String getProductId() {
        return productId;
    }

    public String getName() {
        return name;
    }

    public int getQuantity() {
        return quantity;
    }

    public double getUnitPrice() {
        return unitPrice;
    }
}
```

```
}

    public double getTotalPrice() {
        return unitPrice * quantity - discountAmount;
    }

    public double getDiscountAmount() {
        return discountAmount;
    }

    @Override
    public String toString() {
        return name + " (x" + quantity + ") - RM" +
String.format("%.2f", getTotalPrice())
            + " (Saved RM" + String.format("%.2f",
discountAmount) + ") ";
    }
}
```

12.2. Computer.java :

```
/*
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-de
fault.txt to change this license
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */

public class Computer extends Product {

    public Computer(String id, String name, String brand,
String category,
        int quantity, double costPrice, double
sellingPrice) {
        super(id, name, brand, category, quantity,
costPrice, sellingPrice);
    }

    @Override
    public String toString() {
        return "ID: " + id
            + ", Name: " + name
            + ", Brand: " + brand
            + ", Category: " + category
            + ", Qty: " + quantity
            + ", Cost RM" + costPrice
            + ", Sell RM" + sellingPrice;
    }
}
```

12.3. ComputerManager.java :

```
/*
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-de
fault.txt to change this license
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */

import java.io.*;
import java.util.ArrayList;

public class ComputerManager {

    private ArrayList<Computer> computerList = new
ArrayList<>();
    private String folderPath =
"C:\\Users\\USER\\Desktop\\UUM\\(A242) SEMESTER
6\\(STIAK1113) PROGRAMMING 2\\GROUP
PROJECT\\GroupProject\\src\\textfiles";
    private String defaultFileName = "computers.txt";

    public void addProduct(Computer comp) {
        computerList.add(comp);
        System.out.println("Product added.");
    }

    public void updateProductDetails(String id, String
newName, double newCost, double newSell) {
        for (Computer c : computerList) {
            if (c.getId().equalsIgnoreCase(id)) {
                c.setName(newName);
                c.setCostPrice(newCost);
                c.setSellingPrice(newSell);
                System.out.println("Product updated.");
                return;
            }
        }
        System.out.println("Product not found.");
    }
}
```



```

public boolean deleteProduct(String id) {
    for (Computer c : computerList) {
        if (c.getId().equalsIgnoreCase(id)) {
            computerList.remove(c);
            return true;
        }
    }
    return false;
}

public Computer searchProduct(String id) {
    for (Computer c : computerList) {
        if (c.getId().equalsIgnoreCase(id)) {
            return c;
        }
    }
    return null;
}

public void reduceQuantity(String id, int qty) {
    Computer c = searchProduct(id);
    if (c != null) {
        c.setQuantity(c.getQuantity() - qty);
    }
}

public String displayAll() {
    StringBuilder sb = new StringBuilder();
    sb.append("--- Product List ---\n");
    for (Computer c : computerList) {
        sb.append(String.format("ID: %s | Name: %s | 
Brand: %s | Category: %s | Qty: %d | Cost: RM%.2f | Price: 
RM%.2f\n",
                                c.getId(), c.getName(), c.getBrand(),
c.getCategory(),
                                c.getQuantity(), c.getCostPrice(),
c.getSellingPrice()));
    }
    return sb.toString();
}

public void updateProductAllDetails(String id, String
newName, String newBrand, String newCategory,

```



```

                Double.parseDouble(data[5]),
                Double.parseDouble(data[6])
            );
            computerList.add(comp);
        }
    }

    System.out.println("Product list loaded.");
} catch (IOException e) {
    System.out.println("Error loading products: " +
e.getMessage());
}

}

public void saveToFile() {
    saveToFile(defaultFileName);
}

public void saveToFile(String filename) {
    File dir = new File(folderPath);
    if (!dir.exists()) {
        dir.mkdirs();
    }
    File file = new File(dir, filename);

    try (PrintWriter writer = new PrintWriter(new
FileWriter(file))) {
        for (Computer c : computerList) {
            writer.println(c.getId() + "," + c.getName()
+ "," + c.getBrand() + "," + c.getCategory() + ","
+ c.getQuantity() + "," +
c.getCostPrice() + "," + c.getSellingPrice());
        }
        System.out.println("Product list saved.");
    } catch (IOException e) {
        System.out.println("Error saving products: " +
e.getMessage());
    }
}

}
}

```

12.4. MainApp.java :

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class MainApp {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create manager objects
        ComputerManager cm = new ComputerManager();
        SalesManager sm = new SalesManager();
        MembershipManager mm = new MembershipManager();

        // Load existing data
        cm.loadFromFile();
        sm.loadSalesFromFile();
        mm.loadMembers();

        boolean running = true;

        while (running) {
            System.out.println("\n=== COMPUTER SHOP
MANAGEMENT MENU ===");
            System.out.println("1. Add Product");
            System.out.println("2. Update Product");
            System.out.println("3. Delete Product");
            System.out.println("4. View Products");
            System.out.println("5. Sell Product");
            System.out.println("6. View Sales");
            System.out.println("7. View Profit Summary");
            System.out.println("8. Exit");
            System.out.print("Enter your choice: ");

            int choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1: // Add Product
                    do {
```

```

        System.out.print("Product ID: ");
        String id = sc.nextLine();
        System.out.print("Name: ");
        String name = sc.nextLine();
        System.out.print("Brand: ");
        String brand = sc.nextLine();
        System.out.print("Category: ");
        String category = sc.nextLine();
        System.out.print("Quantity: ");
        int qty = sc.nextInt();
        System.out.print("Cost Price (RM): ");

        double cost = sc.nextDouble();
        System.out.print("Selling Price (RM): ");

        double price = sc.nextDouble();
        sc.nextLine(); // clear newline

        Computer product = new Computer(id, name, brand, category, qty, cost, price);
        cm.addProduct(product);
        cm.saveToFile();

        System.out.print("Add another product? (y/n): ");
        } while (sc.nextLine().equalsIgnoreCase("y"));
        break;

    case 2: // Update Product
        System.out.print("Product ID to update: ");

        String updateId = sc.nextLine();
        System.out.print("New Name: ");
        String newName = sc.nextLine();
        System.out.print("New Cost Price: ");
        double newCost = sc.nextDouble();
        System.out.print("New Selling Price: ");
        double newPrice = sc.nextDouble();
        sc.nextLine();

        cm.updateProductDetails(updateId, newName, newCost, newPrice);

```

```

        cm.saveToFile();
        break;

    case 3: // Delete Product
        System.out.print("Product ID to delete:
");

        String delId = sc.nextLine();
        if (cm.deleteProduct(delId)) {
            System.out.println("Product
deleted.");

            cm.saveToFile();
        } else {
            System.out.println("Product not
found.");
        }
        break;

    case 4: // View Products
        cm.displayAll();
        break;

    case 5: // Sell Product with Cart
        List<Cart_item> cart = new
ArrayList<>();

        double totalDiscount = 0;
        double grandTotal = 0;

        do {
            System.out.print("Enter Product ID
to add to cart: ");

            String sellId = sc.nextLine();
            Computer p =
cm.searchProduct(sellId);

            if (p == null) {
                System.out.println("Product not
found.");

                continue;
            }

            if (p.getQuantity() == 0) {
                System.out.println("Product is
out of stock.");

```

```

        continue;
    }

    System.out.print("Quantity to sell:
");

    int sellQty = sc.nextInt();
    sc.nextLine();

    if (sellQty > p.getQuantity()) {
        System.out.println("Only " +
p.getQuantity() + " available.");
        continue;
    }

    double unitPrice =
p.getSellingPrice();

    double subtotal = unitPrice *
sellQty;

    // Apply base discount
    double baseDiscountRate = 0;
    if (subtotal > 10000) {
        baseDiscountRate = 0.08;
    } else if (subtotal > 5000) {
        baseDiscountRate = 0.05;
    }

    double afterBaseDiscount = subtotal
* (1 - baseDiscountRate);
    double finalAmount =
afterBaseDiscount; // no member discount yet
    double discountAmount = subtotal -
finalAmount;

    Cart_item item = new
Cart_item(p.getId(), p.getName(), sellQty, unitPrice,
discountAmount);

    cart.add(item);
    totalDiscount += discountAmount;
    grandTotal += finalAmount;

    System.out.print("Add another item
to cart? (y/n): ");

```

```

        } while
(sc.nextLine().equalsIgnoreCase("y"));

        // Ask for membership AFTER cart is
finalized

        double memberDiscountRate = 0;
        if (grandTotal > 5000) {
            System.out.print("Enter member phone
(or press Enter to skip): ");
            String phone = sc.nextLine().trim();

            if (!phone.isEmpty()) {
                if (mm.isMember(phone)) {
                    System.out.println("Member
found. Extra 2% discount applied.");
                    memberDiscountRate = 0.02;
                } else {
                    System.out.print("Not a
member. Register now? (y/n): ");
                    String reg = sc.nextLine();
                    if
(reg.equalsIgnoreCase("y")) {
                        mm.registerMember(phone);

                        mm.saveMembers();
                        memberDiscountRate =
0.02;

                        System.out.println("Registered and 2% discount applied.");
                    }
                }
            }
        }

        // Recalculate with membership discount
        double memberSaved = grandTotal *
memberDiscountRate;
        double finalTotal = grandTotal -
memberSaved;
        double overallSaved = totalDiscount +
memberSaved;

        // Display summary

```



```

        System.out.println("\n=== CART SUMMARY
===");

        for (Cart_item item : cart) {
            System.out.println(item);
        }

        System.out.printf("Total saved:
RM%.2f\n", overallSaved);

        System.out.printf("Final total to pay:
RM%.2f\n", finalTotal);

        System.out.print("Proceed to payment?
(y/n): ");

        String confirm = sc.nextLine();
        if (confirm.equalsIgnoreCase("y")) {
            System.out.print("Enter sale date
(YYYY-MM-DD): ");

            String date = sc.nextLine();

            for (Cart_item item : cart) {
                // Apply only per-item discount
                // (not member discount) for recording
                // Get the cost price from the
                product

                Computer product =
cm.searchProduct(item.getProductId());
                double costPrice =
product.getCostPrice();

sm.sellProduct(item.getProductId(), item.getQuantity(),
item.getUnitPrice(), costPrice, date,
item.getDiscountAmount());

cm.reduceQuantity(item.getProductId(), item.getQuantity());
            }

            cm.saveToFile();
            sm.saveSalesToFile();
            System.out.println("Transaction
complete.");
        } else {
            System.out.println("Transaction
canceled.");
        }
    }
}

```

```
        break;

        case 6: // View Sales
            sm.displayAllSales();
            break;

        case 7: // Profit Summary
            sm.viewIncomeStatement();
            break;

        case 8: // Exit
            mm.saveMembers();
            System.out.println("System exiting.
Goodbye!");
            running = false;
            break;

        default:
            System.out.println("Invalid choice.
Please choose 1-8.");
    }
}

sc.close();
}
```

12.5. MembershipManager.java :

```
import java.io.*;
import java.util.ArrayList;

public class MembershipManager {

    private ArrayList<String> memberPhones = new
ArrayList<>();

    private String folderPath =
"C:\\Users\\USER\\Desktop\\UUM\\(A242) SEMESTER
6\\(STIAK1113) PROGRAMMING 2\\GROUP
PROJECT\\GroupProject\\src\\textfiles";
    private String fileName = "members.txt";

    // Load members from file
    public void loadMembers() {
        File dir = new File(folderPath);
        File file = new File(dir, fileName);

        if (file.exists()) {
            try {
                BufferedReader reader = new
BufferedReader(new FileReader(file));
                String line;
                while ((line = reader.readLine()) != null) {
                    if (!line.trim().isEmpty()) {
                        memberPhones.add(line.trim());
                    }
                }
                reader.close();
                System.out.println("Membership loaded.");
            } catch (IOException e) {
                System.out.println("Error loading
membership: " + e.getMessage());
            }
        } else {
            System.out.println("Membership file does not
exist. Creating new when saved.");
        }
    }
}
```

```

// Save members to file
public void saveMembers() {
    File dir = new File(folderPath);
    if (!dir.exists()) {
        dir.mkdirs();
    }

    File file = new File(dir, fileName);

    try {
        PrintWriter writer = new PrintWriter(new
FileWriter(file));
        for (String phone : memberPhones) {
            writer.println(phone);
        }
        writer.close();
        System.out.println("Membership saved.");
    } catch (IOException e) {
        System.out.println("Error saving membership: " +
e.getMessage());
    }
}

// Check using phone number if member exists
public boolean isMember(String phone) {
    return memberPhones.contains(phone);
}

// Register a new member with phone number
public void registerMember(String phone) {
    if (!isMember(phone)) {
        memberPhones.add(phone);
        System.out.println("Phone number registered as
member.");
    } else {
        System.out.println("This phone is already a
member.");
    }
}

// Register a new member with phone and name (overloaded
method)
public void registerMember(String phone, String name) {

```

```
        if (!isMember(phone)) {
            memberPhones.add(phone);
            System.out.println("Phone number " + phone + "
registered as member for " + name);
        } else {
            System.out.println("This phone is already a
member.");
        }
    }

    // For testing: View all members
    public void displayAllMembers() {
        System.out.println("\n--- Registered Members ---");
        for (String phone : memberPhones) {
            System.out.println("• " + phone);
        }
    }
}
```

12.6. Product.java :

```
// Abstract base for any product
public abstract class Product {

    protected String id;
    protected String name;
    protected String brand;
    protected String category;
    protected int quantity;
    protected double costPrice;
    protected double sellingPrice;

    public Product(String id, String name, String brand,
String category,
        int quantity, double costPrice, double
sellingPrice) {
        this.id = id;
        this.name = name;
        this.brand = brand;
        this.category = category;
        this.quantity = quantity;
        this.costPrice = costPrice;
        this.sellingPrice = sellingPrice;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getBrand() {
        return brand;
    }

    public String getCategory() {
        return category;
    }
}
```

```
public int getQuantity() {
    return quantity;
}

public double getCostPrice() {
    return costPrice;
}

public double getSellingPrice() {
    return sellingPrice;
}

public void setBrand(String brand) {
    this.brand = brand;
}

public void setCategory(String category) {
    this.category = category;
}

public void setQuantity(int q) {
    quantity = q;
}

public void setName(String n) {
    name = n;
}

public void setCostPrice(double c) {
    costPrice = c;
}

public void setSellingPrice(double s) {
    sellingPrice = s;
}

@Override
public abstract String toString();
}
```

12.7. Sales.java :

```
public class Sales {

    private String productId;
    private int quantity;
    private double unitPrice;
    private double discountAmount;
    private double costPrice;
    private String date;

    public Sales(String productId, int quantity, double
unitPrice, double costPrice, double discountAmount, String
date) {
        this.productId = productId;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.costPrice = costPrice;
        this.discountAmount = discountAmount;
        this.date = date;
    }

    public String getProductId() {
        return productId;
    }

    public int getQuantity() {
        return quantity;
    }

    public double getUnitPrice() {
        return unitPrice;
    }

    public double getCostPrice() {
        return costPrice;
    }

    public double getTotalCost() {
        return costPrice * quantity;
    }
}
```



```
public double getDiscountAmount() {  
    return discountAmount;  
}  
  
public String getDate() {  
    return date;  
}  
  
public double getFinalTotal() {  
    return (unitPrice * quantity) - discountAmount;  
}  
  
public String toCSV() {  
    return productId + "," + quantity + "," + unitPrice  
+ "," + costPrice + "," + discountAmount + "," + date;  
}  
}
```

12.8. SalesManager.java :

```
import java.io.*;
import java.util.ArrayList;

public class SalesManager {

    private ArrayList<Sales> salesList = new ArrayList<>();
    private String folderPath =
"C:\\Users\\USER\\Desktop\\UUM\\(A242) SEMESTER
6\\(STIAK1113) PROGRAMMING 2\\GROUP
PROJECT\\GroupProject\\src\\textfiles";
    private String fileName = "sales.txt";

    public void sellProduct(String productId, int quantity,
double unitPrice, double costPrice, String date, double
discountAmount) {
        Sales sale = new Sales(productId, quantity,
unitPrice, costPrice, discountAmount, date);
        salesList.add(sale);
        System.out.println("Sale recorded.");
    }

    public void loadSalesFromFile() {
        File file = new File(folderPath, fileName);

        if (!file.exists()) {
            System.out.println("No sales file found. It will
be created when saving.");
            return;
        }

        try (BufferedReader reader = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length == 6) {
                    String id = parts[0];
                    int qty = Integer.parseInt(parts[1]);
                    double price =
Double.parseDouble(parts[2]);
```

```

        double cost =
Double.parseDouble(parts[3]);
        double discount =
Double.parseDouble(parts[4]);
        String date = parts[5];

        salesList.add(new Sales(id, qty, price,
cost, discount, date));
    }
}

    System.out.println("Sales loaded from file.");
} catch (IOException e) {
    System.out.println("Error reading sales file: "
+ e.getMessage());
}
}

    public void saveSalesToFile() {
        File file = new File(folderPath, fileName);

        try (PrintWriter writer = new PrintWriter(new
FileWriter(file))) {
            for (Sales s : salesList) {
                writer.println(s.toCSV());
            }
            System.out.println("Sales saved to file.");
        } catch (IOException e) {
            System.out.println("Error writing sales file: "
+ e.getMessage());
        }
    }

    public String displayAllSales() {
        StringBuilder sb = new StringBuilder();
        sb.append("--- Sales Record ---\n");
        for (Sales s : salesList) {
            sb.append(String.format("Product: %s | Qty: %d |
Unit RM%.2f | Discount RM%.2f | Final Total RM%.2f | Date:
%s\n",

                s.getProductId(), s.getQuantity(),
s.getUnitPrice(),

                s.getDiscountAmount(),
s.getFinalTotal(), s.getDate()));

```

```
    }  
    return sb.toString();  
}  
  
public String viewIncomeStatement() {  
    double totalSales = 0.0;  
    double totalCost = 0.0;  
  
    for (Sales s : salesList) {  
        totalSales += s.getFinalTotal();  
        totalCost += s.getTotalCost();  
    }  
  
    double totalProfit = totalSales - totalCost;  
  
    return String.format("Total Sales: RM%.2f\nTotal  
Profit: RM%.2f", totalSales, totalProfit);  
}  
}
```

12.9. ShopGUI.java :

```
/*
 * Click
nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-de
fault.txt to change this license
 * Click
nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.jav
a to edit this template
 */

/**
 *
 * @author LEONG WEI CHONG
 */
import javax.swing.*;
import java.util.ArrayList;

public class ShopGUI extends javax.swing.JFrame {

    private ComputerManager cm;
    private SalesManager sm;
    private MembershipManager mm;
    private ArrayList<Cart_item> cart = new ArrayList<>();

    /**
     * Creates new form ShopGUI
     */
    public ShopGUI() {
        initComponents();
        cm = new ComputerManager();
        sm = new SalesManager();
        mm = new MembershipManager();

        // Load data when the app starts
        cm.loadFromFile();
        sm.loadSalesFromFile();
        mm.loadMembers();

    }

    public static void main(String[] args) {
        // Set the look and feel for the application
    }
}
```

```

(optional)
        try {
            for (UIManager.LookAndFeelInfo info :
UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {

UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (UnsupportedLookAndFeelException |
ClassNotFoundException | InstantiationException |
IllegalAccessException ex) {

java.util.logging.Logger.getLogger(ShopGUI.class.getName()).
log(java.util.logging.Level.SEVERE, null, ex);
        }

        // Create and display the form
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new ShopGUI().setVisible(true); // Make the
GUI visible
            }
        });
    }

/**
 * This method is called from within the constructor to
initialize the form.
 * WARNING: Do NOT modify this code. The content of this
method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated
Code">//GEN-BEGIN: initComponents
    private void initComponents() {

        jLabel2 = new javax.swing.JLabel();
        jPanel1 = new javax.swing.JPanel();
        jButton2 = new javax.swing.JButton();
        jButton3 = new javax.swing.JButton();

```

```

        jButton4 = new javax.swing.JButton();
        jButton5 = new javax.swing.JButton();
        jButton6 = new javax.swing.JButton();
        jButton7 = new javax.swing.JButton();
        jButton8 = new javax.swing.JButton();
        jLabel1 = new javax.swing.JLabel();
        jButton1 = new javax.swing.JButton();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTextArea1 = new javax.swing.JTextArea();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        jLabel2.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/images/welcomeheader.jpg"))); // NOI18N

jLabel2.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);

        jPanel1.setBackground(new java.awt.Color(255, 204, 153));

        jButton2.setText("2. Update Product");
        jButton2.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton2ActionPerformed(evt);
            }
        });

        jButton3.setText("3. Delete Product");
        jButton3.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton3ActionPerformed(evt);
            }
        });

        jButton4.setText("4. View Products");

```

```
        jButton4.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton4ActionPerformed(evt);
            }
        });

        jButton5.setText("5. Sell Product");
        jButton5.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton5ActionPerformed(evt);
            }
        });

        jButton6.setText("6. View Sales");
        jButton6.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton6ActionPerformed(evt);
            }
        });

        jButton7.setText("7. View Profit Summary");
        jButton7.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton7ActionPerformed(evt);
            }
        });

        jButton8.setText("8. Exit");
        jButton8.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton8ActionPerformed(evt);
            }
        });
    });
}
```



```

        jLabel1.setFont(new java.awt.Font("Segoe UI", 1,
24)); // NOI18N
        jLabel1.setForeground(new java.awt.Color(0, 0, 0));

jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CE
NTER);
        jLabel1.setText("COMPUTER SHOP MANAGEMENT MENU");

        jButton1.setText("1. Add Product");
        jButton1.addActionListener(new
java.awt.event.ActionListener() {
            public void
actionPerformed(java.awt.event.ActionEvent evt) {
                jButton1ActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout jPanel1Layout = new
javax.swing.GroupLayout(jPanel1);
        jPanel1.setLayout(jPanel1Layout);
        jPanel1Layout.setHorizontalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Al
ignment.LEADING)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addGap(151, 151, 151)
                .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 472,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(151, 151, 151)
                .addComponent(jButton8,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            )
        );
    }
}

```

```

javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton6,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton5,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton4,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton3,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton1,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 152,
javax.swing.GroupLayout.PREFERRED_SIZE)))

.addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
    );
    jPanel1Layout.setVerticalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Al
ignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel1,
javax.swing.GroupLayout.PREFERRED_SIZE, 42,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)
            .addComponent(jButton1)
            .addGap(15, 15, 15)
            .addComponent(jButton2)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)
            .addComponent(jButton3)

```

```

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)

        .addComponent(jButton4)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)

        .addComponent(jButton5)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)

        .addComponent(jButton6)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)

        .addComponent(jButton7)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
UNRELATED)

        .addComponent(jButton8)
        .addContainerGap(12, Short.MAX_VALUE))
    );

    jTextArea1.setEditable(false);
    jTextArea1.setColumns(20);
    jTextArea1.setRows(5);
    jScrollPane1.setViewportView(jTextArea1);

    javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

        .addComponent(jScrollPane1)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout
.Alignment.LEADING, false)

            .addComponent(jPanel1,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

            .addComponent(jLabel2,

```

```

javax.swing.GroupLayout.PREFERRED_SIZE, 0, Short.MAX_VALUE))
    );
    layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

.addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
    .addContainerGap()
    .addComponent(jLabel2,
javax.swing.GroupLayout.PREFERRED_SIZE, 153,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
RELATED)
    .addComponent(jPanel1,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.
RELATED)
    .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE, 222,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
    );

    pack();
    setLocationRelativeTo(null);
} // </editor-fold> // GEN-END: initComponents

private void
jButton2ActionPerformed(java.awt.event.ActionEvent evt)
{ // GEN-FIRST: event_jButton2ActionPerformed
    String id = JOptionPane.showInputDialog("Enter
Product ID to update:");
    if (id == null) {
        return;
    }
}

```

```

        // Check if product exists and get current details
        Computer existingProduct = cm.searchProduct(id);
        if (existingProduct == null) {
            JOptionPane.showMessageDialog(this, "Product not
found!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Show current product details
        String currentDetails = String.format(
            "Current Product Details:\n"
            + "ID: %s\n"
            + "Name: %s\n"
            + "Brand: %s\n"
            + "Category: %s\n"
            + "Quantity: %d\n"
            + "Cost Price: RM%.2f\n"
            + "Selling Price: RM%.2f\n\n"
            + "Enter new values :",
            existingProduct.getId(),
            existingProduct.getName(),
            existingProduct.getBrand(),
            existingProduct.getCategory(),
            existingProduct.getQuantity(),
            existingProduct.getCostPrice(),
            existingProduct.getSellingPrice()
        );

        JOptionPane.showMessageDialog(this, currentDetails,
"Current Product Info", JOptionPane.INFORMATION_MESSAGE);

        // Get new name (or keep current)
        String newName = JOptionPane.showInputDialog("Enter
New Product Name:", existingProduct.getName());
        if (newName == null) {
            return;
        }
        if (newName.trim().isEmpty()) {
            newName = existingProduct.getName();
        }

        // Get new brand (or keep current)

```

```

        String newBrand = JOptionPane.showInputDialog("Enter
New Product Brand:", existingProduct.getBrand());
        if (newBrand == null) {
            return;
        }
        if (newBrand.trim().isEmpty()) {
            newBrand = existingProduct.getBrand();
        }

        // Get new category (or keep current)
        String newCategory =
JOptionPane.showInputDialog("Enter New Product Category:",
existingProduct.getCategory());
        if (newCategory == null) {
            return;
        }
        if (newCategory.trim().isEmpty()) {
            newCategory = existingProduct.getCategory();
        }

        // Get new quantity (or keep current)
        String qtyStr = JOptionPane.showInputDialog("Enter
New Quantity:",
String.valueOf(existingProduct.getQuantity()));
        if (qtyStr == null) {
            return;
        }
        int newQuantity = existingProduct.getQuantity();
        if (!qtyStr.trim().isEmpty()) {
            try {
                newQuantity = Integer.parseInt(qtyStr);
                if (newQuantity < 0) {
                    JOptionPane.showMessageDialog(this,
"Quantity cannot be negative!", "Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Invalid
quantity input!", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
        }
    }
}

```

```

        // Get new cost price (or keep current)
        String costStr = JOptionPane.showInputDialog("Enter
New Cost Price:",
String.valueOf(existingProduct.getCostPrice()));
        if (costStr == null) {
            return;
        }
        double newCost = existingProduct.getCostPrice();
        if (!costStr.trim().isEmpty()) {
            try {
                newCost = Double.parseDouble(costStr);
                if (newCost < 0) {
                    JOptionPane.showMessageDialog(this,
"Cost price cannot be negative!", "Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Invalid
cost price input!", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        // Get new selling price (or keep current)
        String priceStr = JOptionPane.showInputDialog("Enter
New Selling Price:",
String.valueOf(existingProduct.getSellingPrice()));
        if (priceStr == null) {
            return;
        }
        double newPrice = existingProduct.getSellingPrice();
        if (!priceStr.trim().isEmpty()) {
            try {
                newPrice = Double.parseDouble(priceStr);
                if (newPrice < 0) {
                    JOptionPane.showMessageDialog(this,
"Selling price cannot be negative!", "Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }
            } catch (NumberFormatException ex) {

```

```

        JOptionPane.showMessageDialog(this, "Invalid
selling price input!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}

// Update all fields
existingProduct.setName(newName);
existingProduct.setBrand(newBrand);
existingProduct.setCategory(newCategory);
existingProduct.setQuantity(newQuantity);
existingProduct.setCostPrice(newCost);
existingProduct.setSellingPrice(newPrice);

cm.saveToFile();

// Show updated details
String updatedDetails = String.format(
    "Product Updated Successfully!\n\n"
    + "Updated Product Details:\n"
    + "ID: %s\n"
    + "Name: %s\n"
    + "Brand: %s\n"
    + "Category: %s\n"
    + "Quantity: %d\n"
    + "Cost Price: RM%.2f\n"
    + "Selling Price: RM%.2f",
    existingProduct.getId(),
    existingProduct.getName(),
    existingProduct.getBrand(),
    existingProduct.getCategory(),
    existingProduct.getQuantity(),
    existingProduct.getCostPrice(),
    existingProduct.getSellingPrice()
);

JOptionPane.showMessageDialog(this, updatedDetails,
"Update Complete", JOptionPane.INFORMATION_MESSAGE);
} // GEN-LAST:event_jButton2ActionPerformed

private void
jButton1ActionPerformed(java.awt.event.ActionEvent evt)
{ // GEN-FIRST:event_jButton1ActionPerformed

```



```

        // Add Product with loop
        while (true) {
            String id = JOptionPane.showInputDialog("Enter
Product ID:");
            if (id == null) {
                return; // User cancelled
            }

            // Check if product ID already exists
            if (cm.searchProduct(id) != null) {
                JOptionPane.showMessageDialog(this, "Product
ID already exists! Please use a different ID.", "Error",
JOptionPane.ERROR_MESSAGE);
                continue; // Ask for ID again
            }

            String name = JOptionPane.showInputDialog("Enter
Product Name:");
            if (name == null) {
                return;
            }
            String brand =
JOptionPane.showInputDialog("Enter Product Brand:");
            if (brand == null) {
                return;
            }
            String category =
JOptionPane.showInputDialog("Enter Product Category:");
            if (category == null) {
                return;
            }
            String qtyStr =
JOptionPane.showInputDialog("Enter Quantity:");
            if (qtyStr == null) {
                return;
            }
            String costStr =
JOptionPane.showInputDialog("Enter Cost Price:");
            if (costStr == null) {
                return;
            }
            String priceStr =
JOptionPane.showInputDialog("Enter Selling Price:");

```

```

        if (priceStr == null) {
            return;
        }

        try {
            int qty = Integer.parseInt(qtyStr);
            double cost = Double.parseDouble(costStr);
            double price = Double.parseDouble(priceStr);

            Computer product = new Computer(id, name,
brand, category, qty, cost, price);
            cm.addProduct(product);
            cm.saveToFile();
            JOptionPane.showMessageDialog(this, "Product
Added Successfully!");

            // Ask if user wants to add another product
            int choice =
JOptionPane.showConfirmDialog(this, "Do you want to add
another product?", "Continue", JOptionPane.YES_NO_OPTION);
            if (choice != JOptionPane.YES_OPTION) {
                break; // Exit the loop
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid
numeric input! Please enter valid numbers.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    //GEN-LAST:event_jButton1ActionPerformed

    private void
jButton3ActionPerformed(java.awt.event.ActionEvent evt)
{ //GEN-FIRST:event_jButton3ActionPerformed
        String id = JOptionPane.showInputDialog("Enter
Product ID to delete:");
        if (id == null) {
            return;
        }

        boolean deleted = cm.deleteProduct(id);
        cm.saveToFile();
        if (deleted) {

```

```

        JOptionPane.showMessageDialog(this, "Product
Deleted Successfully!");
    } else {
        JOptionPane.showMessageDialog(this, "Product Not
Found!", "Error", JOptionPane.ERROR_MESSAGE);
    }
} //GEN-LAST:event_jButton3ActionPerformed

private void
jButton4ActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_jButton4ActionPerformed
    String productList = cm.displayAll();
    if (productList == null ||
productList.trim().isEmpty()) {
        jTextArea1.setText("No products available.");
    } else {

        StringBuilder tableFormat = new StringBuilder();

tableFormat.append("=".repeat(120)).append("\n");
        tableFormat.append(String.format("%-8s | %-25s |
%-15s | %-12s | %-5s | %-10s | %-10s\n",
            "ID", "NAME", "BRAND", "CATEGORY",
"QTY", "COST (RM)", "PRICE (RM)"));

tableFormat.append("=".repeat(120)).append("\n");

        String[] lines = productList.split("\n");
        for (String line : lines) {
            if (line.startsWith("ID:")) {

                String[] parts = line.split(" \\| ");
                if (parts.length >= 7) {
                    String id = parts[0].substring(4);
// Remove "ID: "
                    String name = parts[1].substring(6);
// Remove "Name: "
                    String brand =
parts[2].substring(7); // Remove "Brand: "
                    String category =
parts[3].substring(10); // Remove "Category: "
                    String qty = parts[4].substring(5);
// Remove "Qty: "

```

```

        String cost = parts[5].substring(5);
// Remove "Cost: "
        String price =
parts[6].substring(5); // Remove "Price: "

tableFormat.append(String.format("%-8s | %-25s | %-15s |
%-12s | %-5s | %-10s | %-10s\n",
                                id.length() > 8 ?
id.substring(0, 8) : id,
                                name.length() > 25 ?
name.substring(0, 25) : name,
                                brand.length() > 15 ?
brand.substring(0, 15) : brand,
                                category.length() > 12 ?
category.substring(0, 12) : category,
                                qty, cost, price));
    }
}
}
tableFormat.append("=".repeat(120));
jTextArea1.setText(tableFormat.toString());
}
jTextArea1.setCaretPosition(0); // Scroll to top
} // GEN-LAST:event_jButton4ActionPerformed

private void
jButton5ActionPerformed(java.awt.event.ActionEvent evt)
{ // GEN-FIRST:event_jButton5ActionPerformed
    // Sell Product functionality
    cart.clear();

    java.util.HashMap<String, Integer>
reservedQuantities = new java.util.HashMap<>();

    while (true) {
        String viewList = cm.displayAll();
        /*
        JOptionPane.showMessageDialog(this,
            (viewList == null || viewList.isEmpty()) ?
"No products available." : viewList,
            "Product List",
            JOptionPane.INFORMATION_MESSAGE);

```

```

        */

        String productId =
JOptionPane.showInputDialog("Enter Product ID to sell:");
        if (productId == null) {
            return; // User cancelled
        }

        Computer prod = cm.searchProduct(productId);
        if (prod == null) {
            JOptionPane.showMessageDialog(this, "Product
not found!", "Error", JOptionPane.ERROR_MESSAGE);
            continue;
        }

        // Calculate available quantity (original
quantity minus what's already reserved in cart)
        int reservedQty =
reservedQuantities.getDefault(productId, 0);
        int availableQty = prod.getQuantity() -
reservedQty;

        if (availableQty <= 0) {
            JOptionPane.showMessageDialog(this, "Product
is out of stock or all available quantity is already in your
cart!", "Error", JOptionPane.ERROR_MESSAGE);
            continue;
        }

        String quantityStr =
JOptionPane.showInputDialog("Enter Quantity to Sell
(Available: " + availableQty + "):");
        if (quantityStr == null) {
            return;
        }

        String discountStr =
JOptionPane.showInputDialog("Enter Discount Amount (RM):");
        if (discountStr == null) {
            return;
        }

        try {

```

```

        int quantity =
Integer.parseInt(quantityStr);
        double discount =
Double.parseDouble(discountStr);

        if (quantity <= 0 || quantity >
availableQty) {
            JOptionPane.showMessageDialog(this,
"Invalid quantity! Please enter a valid quantity between 1
and " + availableQty, "Error", JOptionPane.ERROR_MESSAGE);
            continue;
        }

        // Check if this product is already in cart
- update quantity instead of adding new item
        boolean productExistsInCart = false;
        for (Cart_item existingItem : cart) {
            if
(existingItem.getProductId().equals(productId)) {
                // Update existing cart item
                int newQuantity =
existingItem.getQuantity() + quantity;
                double newDiscount =
existingItem.getDiscountAmount() + discount;

                // Create new cart item with updated
values

                Cart_item updatedItem = new
Cart_item(prod.getId(), prod.getName(), newQuantity,
prod.getSellingPrice(), newDiscount);

                // Remove old item and add updated
one

                cart.remove(existingItem);
                cart.add(updatedItem);

                productExistsInCart = true;
                break;
            }
        }

        if (!productExistsInCart) {
            // Add new item to cart

```

```

        cart.add(new Cart_item(prod.getId(),
prod.getName(), quantity, prod.getSellingPrice(),
discount));
    }

    // Update reserved quantities
    reservedQuantities.put(productId,
reservedQuantities.getDefault(productId, 0) + quantity);

    int choice =
JOptionPane.showConfirmDialog(this, "Add another product to
cart?", "Continue", JOptionPane.YES_NO_OPTION);
    if (choice != JOptionPane.YES_OPTION) {
        break; // Exit the loop
    }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Invalid
numeric input! Please enter valid numbers.", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

if (cart.isEmpty()) {
    return; // No items in cart
}

String date = JOptionPane.showInputDialog("Enter
Sale Date (e.g., 2025-06-23):");
if (date == null) {
    return;
}

// Check membership
String phone = JOptionPane.showInputDialog("Enter
phone number to check membership (or press Cancel to
skip):");

boolean isMember = false;
if (phone != null && !phone.trim().isEmpty()) {
    isMember = mm.isMember(phone);
    if (!isMember) {
        int regChoice =
JOptionPane.showConfirmDialog(this, "Not a member. Would you
like to register now?", "Membership Registration",

```

```

JOptionPane.YES_NO_OPTION);
        if (regChoice == JOptionPane.YES_OPTION) {
            String name =
JOptionPane.showInputDialog("Enter customer name:");
            if (name != null &&
!name.trim().isEmpty()) {
                mm.registerMember(phone, name);
                mm.saveMembers();
                isMember = true;
                JOptionPane.showMessageDialog(this,
"Membership registered successfully! 10% discount applied.",
"Success", JOptionPane.INFORMATION_MESSAGE);
            }
        }
        } else {
            JOptionPane.showMessageDialog(this, "Member
found! 10% discount applied.", "Membership",
JOptionPane.INFORMATION_MESSAGE);
        }
    }

    // Process cart and calculate total
    double total = 0;
    StringBuilder receipt = new StringBuilder();
    receipt.append("=== SALES RECEIPT ===\n");
    receipt.append("Date: ").append(date).append("\n");
    if (isMember) {
        receipt.append("Member Phone:
").append(phone).append("\n");
    }
    receipt.append("\n--- Items ---\n");

    for (Cart_item item : cart) {
        cm.reduceQuantity(item.getProductId(),
item.getQuantity());
        double discount = item.getDiscountAmount();
        if (isMember) {
            discount += item.getUnitPrice() *
item.getQuantity() * 0.10; // 10% extra discount for members
        }
        // Get the cost price from the product
        Computer product =
cm.searchProduct(item.getProductId());

```



```

        double costPrice = product.getCostPrice();
        sm.sellProduct(item.getProductId(),
item.getQuantity(), item.getUnitPrice(), costPrice, date,
discount);

        double itemTotal = (item.getUnitPrice() *
item.getQuantity()) - discount;
        total += itemTotal;

        receipt.append(String.format("%s (x%d) -
RM%.2f\n", item.getName(), item.getQuantity(), itemTotal));
        if (discount > 0) {
            receipt.append(String.format(" Discount:
RM%.2f\n", discount));
        }
    }

    receipt.append("\n--- Summary ---\n");
    receipt.append(String.format("Total Amount: RM%.2f",
total));

    cm.saveToFile();
    sm.saveSalesToFile();

    JOptionPane.showMessageDialog(this,
receipt.toString(), "Checkout Complete",
JOptionPane.INFORMATION_MESSAGE);
    } //GEN-LAST:event_jButton5ActionPerformed

    private void
jButton6ActionPerformed(java.awt.event.ActionEvent evt)
{ //GEN-FIRST:event_jButton6ActionPerformed
        String salesList = sm.displayAllSales();
        if (salesList == null || salesList.trim().isEmpty())
        {
            JTextArea1.setText("No sales recorded.");
        } else {
            StringBuilder tableFormat = new StringBuilder();
            tableFormat.append("=".repeat(100)).append("\n");
            tableFormat.append(String.format("%-12s | %-5s |
%-10s | %-12s | %-12s | %-12s\n",
"PRODUCT ID", "QTY", "UNIT (RM)",
"DISCOUNT (RM)", "TOTAL (RM)", "DATE"));

```

```

tableFormat.append("=".repeat(100)).append("\n");

    String[] lines = salesList.split("\n");
    for (String line : lines) {
        if (line.startsWith("Product:")) {
            try {

                String productId = "";
                String qty = "";
                String unit = "";
                String discount = "";
                String total = "";
                String date = "";

                String[] parts = line.split(" \\|\\|
");

                for (String part : parts) {
                    part = part.trim();
                    if (part.startsWith("Product:
")) {

                        productId =
part.substring(9);

                    } else if (part.startsWith("Qty:
")) {

                        qty = part.substring(5);
                    } else if (part.startsWith("Unit
RM")) {

                        unit = part.substring(7);
                    } else if
(part.startsWith("Discount RM")) {

                        discount =
part.substring(11);

                    } else if
(part.startsWith("Final Total RM")) {

                        total = part.substring(14);
                    } else if
(part.startsWith("Date: ")) {

                        date = part.substring(6);

                    }

                }
            }
        }
    }
}

```

```

tableFormat.append(String.format("%-12s | %-5s | %-10s |
%-12s | %-12s | %-12s\n",
                                productId.length() > 12 ?
productId.substring(0, 12) : productId,
                                qty, unit, discount, total,
date));

        } catch (Exception e) {

tableFormat.append(line).append("\n");
        }
    }
    tableFormat.append("=".repeat(100));
    JTextArea1.setText(tableFormat.toString());
}
JTextArea1.setCaretPosition(0); // Scroll to top
} // GEN-LAST:event_jButton6ActionPerformed

private void
jButton7ActionPerformed(java.awt.event.ActionEvent evt)
{ // GEN-FIRST:event_jButton7ActionPerformed
    String profitSummary = sm.viewIncomeStatement();

    StringBuilder summaryFormat = new StringBuilder();
    summaryFormat.append("=".repeat(50)).append("\n");
    summaryFormat.append("                PROFIT SUMMARY REPORT
\n");
    summaryFormat.append("=".repeat(50)).append("\n");
    summaryFormat.append("Date Generated:
").append(java.time.LocalDate.now()).append("\n");
    summaryFormat.append("Time Generated:
").append(java.time.LocalDateTime.now().toString().substring(0,
8)).append("\n");
    summaryFormat.append("-".repeat(50)).append("\n");
    summaryFormat.append(profitSummary).append("\n");
    summaryFormat.append("-".repeat(50)).append("\n");
    // summaryFormat.append("Note: This shows total
revenue from sales\n");
    // summaryFormat.append("(Profit calculation would
require cost analysis)\n");
    // summaryFormat.append("=".repeat(50));

```

```

        jTextArea1.setText(summaryFormat.toString());
        jTextArea1.setCaretPosition(0); // Scroll to top
    } //GEN-LAST:event_jButton7ActionPerformed

    private void
jButton8ActionPerformed(java.awt.event.ActionEvent evt)
{ //GEN-FIRST:event_jButton8ActionPerformed
        mm.saveMembers();
        System.exit(0);
    } //GEN-LAST:event_jButton8ActionPerformed

    // Variables declaration - do not
modify //GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JButton jButton3;
    private javax.swing.JButton jButton4;
    private javax.swing.JButton jButton5;
    private javax.swing.JButton jButton6;
    private javax.swing.JButton jButton7;
    private javax.swing.JButton jButton8;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextArea jTextArea1;
    // End of variables declaration //GEN-END:variables
}

```

12.10. ShopGUI.form

```
<?xml version="1.0" encoding="UTF-8" ?>

<Form version="1.3" maxVersion="1.9"
type="org.netbeans.modules.form.forminfo.JFrameFormInfo">
  <Properties>
    <Property name="defaultCloseOperation" type="int"
value="3"/>
  </Properties>
  <SyntheticProperties>
    <SyntheticProperty name="formSizePolicy" type="int"
value="1"/>
    <SyntheticProperty name="generateCenter" type="boolean"
value="true"/>
  </SyntheticProperties>
  <AuxValues>
    <AuxValue name="FormSettings_autoResourcing"
type="java.lang.Integer" value="0"/>
    <AuxValue name="FormSettings_autoSetComponentName"
type="java.lang.Boolean" value="false"/>
    <AuxValue name="FormSettings_generateFQN"
type="java.lang.Boolean" value="true"/>
    <AuxValue name="FormSettings_generateMnemonicsCode"
type="java.lang.Boolean" value="false"/>
    <AuxValue name="FormSettings_i18nAutoMode"
type="java.lang.Boolean" value="false"/>
    <AuxValue name="FormSettings_layoutCodeTarget"
type="java.lang.Integer" value="1"/>
    <AuxValue name="FormSettings_listenerGenerationStyle"
type="java.lang.Integer" value="0"/>
    <AuxValue name="FormSettings_variablesLocal"
type="java.lang.Boolean" value="false"/>
    <AuxValue name="FormSettings_variablesModifier"
type="java.lang.Integer" value="2"/>
  </AuxValues>

  <Layout>
    <DimensionLayout dim="0">
      <Group type="103" groupAlignment="0" attributes="0">
        <Component id="jScrollPane1" alignment="0"
max="32767" attributes="0"/>
        <Group type="103" groupAlignment="0" max="-2"
```

```

attributes="0">
    <Component id="jPanel1" max="32767"
attributes="0"/>
    <Component id="jLabel2" pref="0" max="32767"
attributes="0"/>
    </Group>
</Group>
</DimensionLayout>
<DimensionLayout dim="1">
    <Group type="103" groupAlignment="0" attributes="0">
        <Group type="102" alignment="1" attributes="0">
            <EmptySpace max="-2" attributes="0"/>
            <Component id="jLabel2" min="-2" pref="153"
max="-2" attributes="0"/>
            <EmptySpace max="-2" attributes="0"/>
            <Component id="jPanel1" min="-2" max="-2"
attributes="0"/>
            <EmptySpace max="-2" attributes="0"/>
            <Component id="jScrollPane1" min="-2"
pref="222" max="-2" attributes="0"/>
            <EmptySpace max="32767" attributes="0"/>
        </Group>
    </Group>
</DimensionLayout>
</Layout>
<SubComponents>
    <Component class="javax.swing.JLabel" name="jLabel2">
        <Properties>
            <Property name="icon" type="javax.swing.Icon"
editor="org.netbeans.modules.form.editors2.IconEditor">
                <Image iconType="3"
name="/images/welcomeheader.jpg"/>
            </Property>
            <Property name="verticalAlignment" type="int"
value="3"/>
        </Properties>
    </Component>
    <Container class="javax.swing.JPanel" name="jPanel1">
        <Properties>
            <Property name="background" type="java.awt.Color"
editor="org.netbeans.beaninfo.editors.ColorEditor">
                <Color blue="99" green="cc" red="ff" type="rgb"/>
            </Property>

```

```

        </Properties>

        <Layout>
            <DimensionLayout dim="0">
                <Group type="103" groupAlignment="0"
attributes="0">
                    <Group type="102" alignment="0"
attributes="0">
                        <EmptySpace max="-2" attributes="0"/>
                        <Group type="103" groupAlignment="0"
attributes="0">
                            <Component id="jLabel1" alignment="0"
min="-2" pref="472" max="-2" attributes="0"/>
                            <Group type="102" alignment="0"
attributes="0">
                                <EmptySpace min="-2" pref="151"
max="-2" attributes="0"/>
                                <Group type="103"
groupAlignment="0" max="-2" attributes="0">
                                    <Component id="jButton8"
max="32767" attributes="0"/>
                                    <Component id="jButton7"
max="32767" attributes="0"/>
                                    <Component id="jButton6"
max="32767" attributes="0"/>
                                    <Component id="jButton5"
max="32767" attributes="0"/>
                                    <Component id="jButton4"
max="32767" attributes="0"/>
                                    <Component id="jButton3"
max="32767" attributes="0"/>
                                    <Component id="jButton1"
max="32767" attributes="0"/>
                                    <Component id="jButton2"
alignment="0" min="-2" pref="152" max="-2" attributes="0"/>
                                </Group>
                            </Group>
                        </Group>
                    <EmptySpace max="32767" attributes="0"/>
                </Group>
            </DimensionLayout>
            <DimensionLayout dim="1">

```

```
<Group type="103" groupAlignment="0"
attributes="0">
    <Group type="102" alignment="0"
attributes="0">
        <EmptySpace max="-2" attributes="0"/>
        <Component id="jLabel1" min="-2" pref="42"
max="-2" attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton1" min="-2" max="-2"
attributes="0"/>
        <EmptySpace min="-2" pref="15" max="-2"
attributes="0"/>
        <Component id="jButton2" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton3" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton4" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton5" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton6" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton7" min="-2" max="-2"
attributes="0"/>
        <EmptySpace type="unrelated" max="-2"
attributes="0"/>
        <Component id="jButton8" min="-2" max="-2"
attributes="0"/>
        <EmptySpace pref="12" max="32767"
attributes="0"/>
    </Group>
</Group>
```



```
        </DimensionLayout>
    </Layout>
    <SubComponents>
        <Component class="javax.swing.JButton"
name="jButton2">
            <Properties>
                <Property name="text" type="java.lang.String"
value="2. Update Product"/>
            </Properties>
            <Events>
                <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton2ActionPerformed"/>
            </Events>
        </Component>
        <Component class="javax.swing.JButton"
name="jButton3">
            <Properties>
                <Property name="text" type="java.lang.String"
value="3. Delete Product"/>
            </Properties>
            <Events>
                <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton3ActionPerformed"/>
            </Events>
        </Component>
        <Component class="javax.swing.JButton"
name="jButton4">
            <Properties>
                <Property name="text" type="java.lang.String"
value="4. View Products"/>
            </Properties>
            <Events>
                <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton4ActionPerformed"/>
            </Events>
        </Component>
        <Component class="javax.swing.JButton"
```

```
name="jButton5">
    <Properties>
        <Property name="text" type="java.lang.String"
value="5. Sell Product"/>
    </Properties>
    <Events>
        <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton5ActionPerformed"/>
    </Events>
</Component>
<Component class="javax.swing.JButton"
name="jButton6">
    <Properties>
        <Property name="text" type="java.lang.String"
value="6. View Sales"/>
    </Properties>
    <Events>
        <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton6ActionPerformed"/>
    </Events>
</Component>
<Component class="javax.swing.JButton"
name="jButton7">
    <Properties>
        <Property name="text" type="java.lang.String"
value="7. View Profit Summary"/>
    </Properties>
    <Events>
        <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton7ActionPerformed"/>
    </Events>
</Component>
<Component class="javax.swing.JButton"
name="jButton8">
    <Properties>
        <Property name="text" type="java.lang.String"
value="8. Exit"/>
```

```

        </Properties>
        <Events>
            <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton8ActionPerformed"/>
        </Events>
    </Component>
    <Component class="javax.swing.JLabel"
name="jLabel1">
        <Properties>
            <Property name="font" type="java.awt.Font"
editor="org.netbeans.beaninfo.editors.FontEditor">
                <Font name="Segoe UI" size="24" style="1"/>
            </Property>
            <Property name="foreground"
type="java.awt.Color"
editor="org.netbeans.beaninfo.editors.ColorEditor">
                <Color blue="0" green="0" red="0" type="rgb"/>
            </Property>
            <Property name="horizontalAlignment" type="int"
value="0"/>
            <Property name="text" type="java.lang.String"
value="COMPUTER SHOP MANAGEMENT MENU"/>
        </Properties>
    </Component>
    <Component class="javax.swing.JButton"
name="jButton1">
        <Properties>
            <Property name="text" type="java.lang.String"
value="1. Add Product"/>
        </Properties>
        <Events>
            <EventHandler event="actionPerformed"
listener="java.awt.event.ActionListener"
parameters="java.awt.event.ActionEvent"
handler="jButton1ActionPerformed"/>
        </Events>
    </Component>
</SubComponents>
</Container>
<Container class="javax.swing.JScrollPane"
name="jScrollPane1">

```

```
<AuxValues>
  <AuxValue name="autoScrollPane"
type="java.lang.Boolean" value="true"/>
</AuxValues>

<Layout
class="org.netbeans.modules.form.compat2.layouts.support.JSc
rollPaneSupportLayout"/>
  <SubComponents>
    <Component class="javax.swing.JTextArea"
name="jTextArea1">
      <Properties>
        <Property name="editable" type="boolean"
value="false"/>
        <Property name="columns" type="int" value="20"/>
        <Property name="rows" type="int" value="5"/>
      </Properties>
    </Component>
  </SubComponents>
</Container>
</SubComponents>
</Form>
```

12.11. welcomeheader.jpg



Image 10.0 : Welcome Header